Babeș-Bolyai University of Cluj-Napoca
Faculty of Mathematics and Computer Science
Computer Science Specialization

A Privacy-Preserving, Federated Anomaly Detection Framework for Distributed Microservice Architectures

Bachelor's Thesis
Submitted in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Science

Scientific Coordinator
Prof. Dr. Sergiu Adrian Dărăbanț

Student
Moghioroș Eric

May 14, 2026

Universitatea Babeș-Bolyai din Cluj-Napoca
Facultatea de Matematică și Informatică
Specializarea Informatică

Un Cadru Federat de Detecție a Anomaliilor cu Păstrarea Confidențialității pentru Arhitecturi Distribuite de Microservicii

Lucrare de Licență

Coordonator Științific
Prof. Dr. Sergiu Adrian Dărăbanț

Absolvent
Moghioroș Eric

May 14, 2026

Abstract

Modern distributed systems, composed of microservices and heterogeneous components, are increasingly complex and prone to performance anomalies that can compromise reliability and service quality. This thesis presents a comprehensive anomaly detection framework designed for large-scale distributed systems, integrating real-time edge detection, federated learning for collaborative model training, causal graph-based root cause analysis, and adaptive, performance-aware thresholding. By combining localized anomaly detection at the edge with global model aggregation via the FedAvg algorithm, the framework enhances detection accuracy while preserving data privacy through Differential Privacy. The incorporation of causal graphs enables precise identification of root causes, facilitating rapid remediation and reducing alert latency. Adaptive thresholding powered by Q-learning dynamically adjusts detection sensitivity according to system performance metrics, improving responsiveness under varying workloads. Experimental evaluation demonstrates that the proposed approach achieves high detection fidelity, low false-positive rates, and scalable performance across distributed architectures, offering a robust solution for proactive system reliability management.

Contents

1 Introduction **8**

1.1 Motivation 8

1.2 Problem Statement 8

1.3 Research Objectives 9

1.4 Research Contributions 10

2 Background and Related Work **12**

2.1 Distributed Microservice Systems 12

2.2 Anomaly Detection for Telemetry 13

2.3 Edge and Federated Learning 13

2.4 Differential Privacy 14

2.5 Root Cause Analysis and Adaptive Thresholding 15

2.6 Research Gap 15

3 LSTM-Autoencoder Model Architecture **17**

3.1 Recurrent Neural Networks and Their Limitations 17

3.2 Long Short-Term Memory (LSTM) 18

3.2.1 LSTM Architecture 18

3.2.2 LSTM for Anomaly Detection 19

3.3 Autoencoders 19

3.3.1 Autoencoder Architecture 19

3.3.2 Autoencoders for Anomaly Detection 19

3.4 LSTM-Autoencoder Architecture 20

3.4.1 Combined Architecture 20

3.5 Bidirectional LSTM 20

3.5.1 Model Configuration 21
3.6 Training Procedure 21

4 Framework Methodology **23**
4.1 Edge Telemetry Pipeline 23
4.2 LSTM-Autoencoder Detector 24
4.3 Federated Learning Protocol 25
4.4 Differential Privacy Mechanism 26
4.5 Communication Reduction 27
4.6 Causal Root Cause Analysis 27
4.7 Adaptive Thresholding with Q-Learning 28
4.8 Algorithmic Workflow 30

5 System Design and Implementation **32**
5.1 End-to-End Architecture 32
5.2 Data Flow 33
5.3 Component Design 33
5.3.1 Edge Agent 33
5.3.2 Central Coordinator 34
5.3.3 Trace Collector and RCA Engine 35
5.4 Configuration and Deployment 35
5.5 Reliability and Fault Tolerance 36
5.6 Implementation Stack 37

6 Experimental Evaluation **38**
6.1 Experimental Setup 38
6.1.1 Hardware, Software, and Datasets 38
6.1.2 Baselines and Evaluation Metrics 39
6.2 Anomaly Detection Performance 40
6.2.1 Cold-Start Convergence 42
6.3 Differential Privacy and Gradient Clipping Analysis 43
6.3.1 Privacy–Accuracy Trade-off 43
6.3.2 Gradient Clipping Sensitivity 43
6.4 Communication Overhead 44

6.4.1 Adaptive Compression 45
6.5 Root Cause Analysis Accuracy 46
6.6 Adaptive Thresholding and Q-Learning Convergence 47
6.6.1 Alert Fatigue Comparison 47
6.6.2 Q-Learning Convergence Analysis 48
6.7 Scalability 48
6.7.1 Ablation Summary 49
6.8 Threats to Validity 49
6.9 Discussion 49
6.9.1 Privacy Guarantee Scope 50

7 Conclusion and Future Work **51**
7.1 Summary of Contributions 51
7.2 Conclusion 51
7.3 Limitations 52
7.4 Future Work 52

References **54**

List of Figures

5.1 High-level architecture of the distributed anomaly detection framework 32

6.1 Detection comparison on SMD (seed 42). The static 3σ baseline (left) fails entirely. The centralized LSTM-AE achieves moderate separation but lower recall. The federated LSTM-AE provides the strongest separation (Table 6.2). 41

6.2 Hyperparameter sweep across six LSTM-AE configurations (seed 42). F1 varies non-monotonically with hidden size: Config E (h=128, L=3) underperforms Config A (h=32, L=1), suggesting that deeper models overfit on the SMD training split. 42

6.3 Cold-start convergence over 100 FL rounds. MSE loss decreases rapidly in the first 20 rounds, but F1 remains near zero until round 50. The lag reflects a threshold calibration delay: the fixed 95th-percentile threshold does not adapt as the reconstruction-error distribution shifts during early training. 42

6.4 Privacy–accuracy trade-off under five DP-SGD noise levels (C=1.0, 200 rounds, seed 42). AUC-ROC degrades gradually, but F1 drops sharply between σ=0.0005 and σ=0.001, marking a phase transition where noise begins to dominate the gradient signal. 43

6.5 Payload size (KB) by serialization and compression. Binary TorchSave is 5.4× smaller than JSON; Zstd L1 yields 61% reduction on text but only 9% on binary. 45

6.6 FL round latency breakdown (10 rounds, seed 42). Local training accounts for >97% of wall-clock time, confirming that communication overhead is not the FL bottleneck for this model size. 45

6.7 RCA accuracy across topology sizes (Wilson 95% CIs). 46

6.8 Threshold trajectories and cumulative alerts (seed 42). The static threshold (blue) accumulates alerts linearly. EWMA (orange) over-triggers due to its smoothing lag. Q-learning (green) initially over-alerts during ε-greedy exploration, then stabilizes after the exploration rate decays below 0.05. 47

6.9 FedAvg aggregation time and memory vs. client count; both scale linearly, confirming the coordinator is not the bottleneck. Clients are simulated in-process; network latency was not emulated. 48

List of Tables

3.1 LSTM-AE architecture configuration 21

4.1 Fault-aware behavior in the federated training protocol 26

4.2 Reward coefficients for Q-learning thresholding (Eq. 4.18). The asymmetry $\gamma_{\mathrm{FN}} > \beta$ reflects the operational preference that missed anomalies are more harmful than false alarms. 29

5.1 Coordinator persistence records required for recovery and auditing. 34

5.2 Recovery behavior for critical framework state 36

6.1 RCA evaluation topologies. The Alibaba topology is a synthetic graph built from Alibaba's published degree distribution, not a direct trace replay. 39

6.2 Detection performance on SMD (Config C, 100 epochs). The primary result is from the pre-trained checkpoint (seed 42). A 3-seed retraining (seeds 42, 123, 7) yields F1 = 0.743 ± 0.077 and AUC-ROC = 0.932 ± 0.017, confirming that AUC-ROC is stable while F1 is sensitive to threshold calibration. 40

6.3 LSTM-AE hyperparameter sweep on SMD (seed 42). Config C (h=128, L=2) achieves the best F1 (0.839) and recall (0.868). Config A reaches similar F1 with a smaller model, suggesting that hidden size 32 may suffice for simpler deployments. 41

6.4 DP-SGD utility after 200 rounds with C=1.0; F1 drops sharply at σ=0.001. 43

6.5 Clipping sensitivity at σ=0.001; smaller C recovers utility by reducing the absolute noise σC. 44

6.6 Federated update payload size and compression latency; binary serialization is 5.4× smaller than text formats. 44

6.7 Adaptive compression policy: Zstd saves bandwidth, while LZ4 reduces latency under high CPU load. 45

6.8 RCA ranking accuracy on Train-Ticket and Alibaba-derived topologies (100 and 50 fault-injection events respectively, Wilson 95% CIs). The framework outperforms random selection on Top-1 for Train-Ticket (42% vs 35%) but shows limited advantage on the larger Alibaba topology. 46
6.9 Alert fatigue comparison (10 020-minute SMD stream, seed 42). Q-learning produces a high alert burst during early exploration (328 alerts by minute 360) but converges to the lowest final alert count (28) with FPR = 0.016. 47
6.10 Q-learning state evolution (seed 42). The ε-decay from 0.10 to 0.01 confirms the exploration-to-exploitation transition. The persistently negative cumulative reward indicates that the default reward coefficients penalize early exploration too heavily; warm-starting or reward shaping would mitigate this. 48
6.11 FedAvg aggregation scalability up to 1 000 participants. 48

Chapter 1

Introduction

1.1 Motivation

Modern software systems increasingly rely on microservice architectures, in which business capabilities are decomposed into independently deployable services connected through network APIs [38, 13]. This architecture improves scalability and development velocity, but it also makes failures harder to observe and diagnose. A latency spike in one service may propagate through several downstream calls, while the operational evidence is split across metrics, logs, traces, and deployment metadata [47, 34].

Traditional centralized monitoring is poorly suited to this environment. Raw telemetry streams are large, geographically dispersed, and often sensitive. A monitoring system that centralizes all observations may introduce bandwidth bottlenecks, latency, and privacy exposure. At the same time, simple static thresholds are brittle under bursty traffic and non-stationary workloads. These conditions motivate a monitoring framework that performs inference close to the telemetry source, learns collaboratively across nodes, and explains anomalies in terms of service dependencies.

1.2 Problem Statement

This thesis addresses the following problem: how can a distributed microservice platform detect anomalies accurately, preserve the privacy of local telemetry, identify likely root causes, and remain operational under partial failures? More precisely, the system receives a continuous stream of service-level and host-level telemetry from multiple nodes, produces an anomaly decision for each local time window, and attaches operational context to the alert when several services degrade together. The

desired output is not only a binary anomaly label, but also a reproducible anomaly score, a threshold value, the model version that produced the decision, and a ranked list of likely root-cause services when the causal graph contains enough evidence.

The problem is challenging because the monitoring pipeline must balance four competing requirements:

- **Detection quality**: the model must capture temporal and multivariate patterns rather than isolated threshold violations.
- **Privacy**: local telemetry should not be centralized, and model updates should reveal as little as possible about individual samples.
- **Operational usefulness**: anomaly alerts should be linked to probable root causes and should avoid unnecessary alert fatigue.
- **Deployability**: the system must tolerate node dropout, bounded bandwidth, and coordinator restarts.

These requirements create trade-offs: centralizing telemetry improves training visibility but exposes sensitive data; stronger DP protects privacy but can corrupt small gradients after convergence; lower thresholds increase recall but cause alert fatigue. The thesis treats anomaly detection as a systems problem.

The scope is bounded: the framework assumes numerical telemetry sampled at regular intervals and service dependencies recoverable from traces or deployment metadata. It does not attempt Byzantine-robust aggregation or complete privacy guarantees, but evaluates whether a practical edge/federated architecture can improve detection, reduce data movement, and provide useful diagnostics.

1.3 Research Objectives

The objective of this work is to design and evaluate a privacy-preserving, federated anomaly detection framework for distributed microservice systems. The research is organized around five concrete objectives:

1. design an edge-deployable LSTM-Autoencoder (LSTM-AE) for local multivariate anomaly detection [36];

2. train the model collaboratively through Federated Averaging (FedAvg) while applying client-side differential privacy [37, 14, 1];

3. reduce federated communication overhead through binary serialization, Zstandard, LZ4, and adaptive compression;

4. enrich anomaly alerts with causal-graph-based root cause analysis;

5. evaluate adaptive Q-learning thresholding and identify the conditions under which it reduces or fails to reduce alert fatigue.

These objectives are mapped to the thesis evaluation as follows. The first objective is tested through the SMD anomaly-detection benchmark and the LSTM-AE hyperparameter sweep. The second objective is tested through the FedAvg baseline and the differential privacy experiments, where the updated results distinguish between the default clipping configuration ($C = 1.0$) and the tighter clipping regime ($C = 0.01$). The third objective is tested through payload serialization, Zstandard, LZ4, and adaptive compression measurements. The fourth objective is tested through causal-graph RCA on Train-Ticket and Alibaba-scale topologies. The fifth objective is tested through the Q-learning thresholding benchmark (10 020-minute SMD stream, seed 42), which is reported as an exploratory result: the policy eventually reduces final alert volume, but its early exploration remains unstable and therefore requires reward redesign before production use.

The central research questions are therefore:

1. Can a federated LSTM-AE match or improve anomaly detection quality without centralizing raw telemetry?

2. How sensitive is the privacy–utility trade-off to the DP noise multiplier and clipping norm?

3. Does communication compression materially reduce federated update overhead without changing model semantics?

4. Can trace-derived causal graphs provide useful root-cause rankings for multi-service incidents?

5. Under what conditions does adaptive thresholding reduce alert fatigue, and when does it fail?

1.4 Research Contributions

The thesis contributes a complete framework and an empirical evaluation with the following outcomes:

- a federated LSTM-AE that achieves competitive detection performance on the Server Machine Dataset (SMD), outperforming the centralized LSTM-AE baseline (full results in §6.2);
- a differential privacy benchmark showing a sharp utility transition: mild noise ($\sigma \leq 0.0005$, $\varepsilon \geq 10$) preserves near-baseline performance, while stronger noise requires careful clipping (§6.3);
- a clipping sensitivity analysis showing that tight clipping recovers detection utility under moderate noise, while also exposing the practical privacy tension created by very small effective noise (§6.3.2);
- a causal graph RCA engine evaluated on Train-Ticket and Alibaba-derived topologies (§6.5);
- a Q-learning thresholding study that reduces final alert volume while motivating warm starts and reward shaping due to unstable early exploration (§6.6);
- a communication and scalability evaluation showing that binary payloads are substantially smaller than text payloads and that FedAvg aggregation scales approximately linearly to 1 000 nodes (§6.4, §6.7).

The novelty of this thesis is therefore system-level rather than component-only. Prior work commonly studies deep anomaly detection, federated optimization, differential privacy, RCA, or adaptive thresholds as separate mechanisms. This work evaluates how these mechanisms behave when they are connected in one edge-deployable monitoring architecture.

This integration is important because the interfaces change the result: threshold calibration changes the usefulness of reconstruction errors, DP clipping changes the utility of federated updates, compression changes deployment feasibility, and RCA changes whether an alert is actionable. The strongest empirical result is the federated LSTM-AE detection performance, while the main cautionary results are the DP clipping tension and the Q-learning cold-start failure. Reporting both positive and negative outcomes is part of the contribution, because it makes the framework reproducible and gives clear directions for future improvement.

Chapter 2

Background and Related Work

2.1 Distributed Microservice Systems

A distributed system is a set of autonomous computing nodes that cooperate through message passing and appear as a coherent system to users [50, 12]. Microservices apply this principle at the application level by splitting a system into small services, each owned, deployed, and scaled independently [38, 13]. The same decomposition that enables rapid deployment also creates operational complexity: service instances are ephemeral, dependencies change frequently, and a user-visible symptom may be several network calls away from its original cause.

Reliability in such systems is constrained by partial failure. The CAP theorem, introduced by Brewer and formalized by Gilbert and Lynch, states that a distributed data store cannot simultaneously guarantee consistency, availability, and partition tolerance under network partitions [6, 20]. Monitoring systems usually prioritize availability and partition tolerance: local anomaly detection should continue even when the global coordinator is temporarily unreachable. This choice accepts short-lived model or graph staleness in exchange for continuous visibility.

Observability is built from metrics, logs, and traces [29]. Metrics describe numerical health indicators such as CPU utilization, latency percentiles, memory pressure, request rate, and error rate. Logs provide event-level context. Traces reconstruct request paths across service boundaries and make dependency-aware diagnosis possible [47]. An effective anomaly detection framework must combine these signals: metrics trigger detection, traces provide topology, and historical behavior provides the baseline against which abnormality is defined.

Existing observability platforms typically centralize raw telemetry for analysis, which introduces bandwidth bottlenecks and privacy exposure at scale. This thesis addresses that limitation through

edge-local detection (§4.2) and federated model training (§4.3), so that raw data remains at the source while model intelligence is shared.

2.2 Anomaly Detection for Telemetry

Classical anomaly detection uses static limits, z-scores, rolling statistics, or Exponentially Weighted Moving Averages (EWMA) [4, 9]. These approaches are inexpensive and interpretable, but they are weak under multivariate and non-stationary telemetry. A latency increase may be harmless during a traffic burst, but suspicious when coupled with memory growth and elevated downstream errors.

Deep unsupervised models are widely used when labeled failures are scarce [8, 40]. Autoencoders learn a compressed representation of normal data and use reconstruction error as an anomaly score. For a window $X \in \mathbb{R}^{T \times d}$ and reconstruction $\hat{X}$, the mean squared reconstruction loss is:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{Td} \sum_{t=1}^{T} \sum_{j=1}^{d} \left(X_{tj} - \hat{X}_{tj} \right)^2 . \tag{2.1}$$

Long Short-Term Memory (LSTM) networks are suitable for telemetry because they capture temporal dependencies more effectively than vanilla recurrent networks [25, 36]. An LSTM-Autoencoder combines an LSTM encoder, a compact latent representation, and an LSTM decoder. Windows that do not match learned normal dynamics produce larger reconstruction errors, which can be converted into anomaly decisions through a threshold.

However, most LSTM-based detectors assume centralized training or offline threshold calibration. This thesis embeds an LSTM-AE within a federated training loop (§4.3) and pairs it with adaptive Q-learning thresholding (§4.7), addressing both limitations.

2.3 Edge and Federated Learning

Edge computing places computation near the data source, reducing latency, bandwidth use, and raw-data exposure [45, 43]. In this thesis, each monitored service is paired with an edge agent that performs local preprocessing, LSTM-AE inference, and local training. The central coordinator aggregates models rather than raw telemetry.

Federated learning (FL) trains a shared model across clients without centralizing their datasets [37, 28]. Given clients $k = 1, \ldots, K$ with local datasets $\mathcal{D}_k$, the global objective is:

$$\min_{\theta} F(\theta) = \sum_{k=1}^{K} \frac{|\mathcal{D}_k|}{\sum_{j=1}^{K} |\mathcal{D}_j|} F_k(\theta). \tag{2.2}$$

FedAvg updates the global model by averaging locally trained client weights:

$$\theta_{t+1} = \sum_{k \in \mathcal{P}_t} \frac{n_k}{\sum_{j \in \mathcal{P}_t} n_j} \theta_t^{(k)}, \tag{2.3}$$

where $\mathcal{P}_t$ is the set of clients that participate in round t. This partial-participation formulation is important in edge deployments, where slow nodes, late joiners, and temporary disconnections are common.

Existing FL deployments typically assume reliable network connectivity and ignore the communication cost of model updates. This thesis addresses both limitations through adaptive compression (§4.5) and pull-based model distribution (§4.3).

2.4 Differential Privacy

Federated learning reduces raw-data movement, but model updates can still leak information through membership inference or model inversion attacks [46, 17]. Differential privacy (DP) limits the influence of any single training sample on the released update [14, 15]. A randomized mechanism $\mathcal{M}$ satisfies (ε, δ)-DP if, for neighboring datasets D and D' differing in one sample and all output sets S:

$$\Pr[\mathcal{M}(D) \in S] \leq e^{\varepsilon} \Pr[\mathcal{M}(D') \in S] + \delta. \tag{2.4}$$

In DP-SGD, per-sample gradients are clipped to a norm C and perturbed with Gaussian noise:

$$\tilde{g} = \frac{1}{B} \sum_{i=1}^{B} g_i \min\left(1, \frac{C}{\|g_i\|_2}\right) + \mathcal{N}(0, \sigma^2 C^2 I). \tag{2.5}$$

The experimental chapters show why both parameters matter. Increasing σ improves privacy but can damage the gradient signal; decreasing C limits sensitivity and also reduces the effective noise magnitude $\sigma \cdot C$. This interaction is central to the updated privacy benchmark in this thesis.

2.5 Root Cause Analysis and Adaptive Thresholding

An anomaly detector answers whether a service looks abnormal; operators also need to know where the problem likely started. Graph-based RCA systems build service dependency graphs from distributed traces and rank likely causes using structural and temporal evidence [32, 53, 18]. PageRank-inspired scoring is useful because an upstream service can explain anomalies several hops downstream [39, 7].

Thresholding remains a separate challenge. Static thresholds are simple, while EWMA and rolling statistics adapt to local drift. Reinforcement learning frames threshold selection as a sequential control problem in which an agent observes recent detection outcomes and adjusts the decision boundary. Q-learning estimates the long-term value of action u in state x through the Bellman update [51, 49]:

$$Q(x_t, u_t) \leftarrow Q(x_t, u_t) + \eta \left[R_{t+1} + \gamma \max_{u'} Q(x_{t+1}, u') - Q(x_t, u_t) \right]. \tag{2.6}$$

The literature motivates Q-learning for adaptive thresholding [35], but this thesis treats it empirically rather than assuming success. The final evaluation shows that a tabular Q-learning tuner can reduce final alert volume after extended interaction, but its early exploration and reward dynamics remain unstable enough to require redesign before production use.

The literature motivates Q-learning for threshold adaptation, but most studies assume stationary reward signals and do not evaluate cold-start behavior in imbalanced anomaly streams. This thesis treats Q-learning thresholding as an empirical question (§6.6) and reports both the eventual stabilization and the cold-start failure as equally important outcomes.

2.6 Research Gap

Existing work usually studies the pieces of the monitoring problem separately. Deep anomaly detectors focus on detection accuracy, federated learning studies distributed optimization, differential privacy studies formal leakage bounds, and RCA systems study graph-based diagnosis. Operational monitoring, however, requires these mechanisms to interact. A detector with high offline AUC is insufficient if thresholds create noisy alerts, if training centralizes raw telemetry, or if alerts do not identify a likely service-level cause.

The state of the art therefore provides strong building blocks but leaves an integration gap. LSTM-based detectors are effective for temporal telemetry, but many assume centralized training or offline threshold calibration. Federated learning avoids raw-data centralization, but FL studies often

report optimization accuracy rather than alert quality, RCA usefulness, or communication feasibility. DP-SGD provides formal protection, but its practical utility depends on clipping, sampling, and convergence dynamics that are often hidden behind a single ε value. Graph-based RCA improves diagnosis, but is usually evaluated separately from the detector that produces the anomalous service set.

This thesis addresses that intersection by evaluating edge-local detection, FedAvg, DP-SGD, compression, causal RCA, and adaptive thresholding under one coherent architecture. Its contribution is not that each mechanism exists in isolation, but that their combined behavior is measured as a deployable monitoring pipeline, including the negative Q-learning cold-start result and the updated DP clipping sensitivity benchmark.

The background surveyed here identifies the building blocks and their individual limitations. Chapter 3 narrows the focus to the LSTM-Autoencoder, the specific detection architecture used by this framework, and defines its structure, training procedure, and the design choices that make it suitable for edge deployment.

Chapter 3

LSTM-Autoencoder Model Architecture

3.1 Recurrent Neural Networks and Their Limitations

Recurrent Neural Networks (RNNs) are a foundational class of neural architectures designed to process sequential data [42, 21]. Unlike feedforward networks, RNNs introduce recurrent connections that allow information to persist across time steps. At each step t, the network combines the current input x_t with the previous hidden state h_{t-1} to compute:

$$h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \tag{3.1}$$

where W_{xh} and W_{hh} are learnable weight matrices, b_h is a bias vector, and σ is a nonlinear activation (typically tanh).

Despite their expressiveness, vanilla RNNs suffer from well-known training instabilities when learning long-range dependencies [5, 41]. During Backpropagation Through Time (BPTT), repeated multiplication of weight matrices across many steps causes gradients to either vanish exponentially or grow without bound. The vanishing gradient problem is particularly harmful, as the error signal decays before it can influence earlier states. Gradient clipping [41] mitigates exploding gradients, but the vanishing gradient problem fundamentally motivates the use of gated architectures such as Long Short-Term Memory networks.

3.2 Long Short-Term Memory (LSTM)

3.2.1 LSTM Architecture

The LSTM network, introduced by Hochreiter and Schmidhuber [25, 22], addresses the vanishing gradient problem through a gating mechanism that regulates information flow within the recurrent cell. An LSTM cell maintains a cell state C_t (long-term memory) and a hidden state h_t (working memory), governed by three multiplicative gates:

- **Forget Gate** (f_t): Determines what to discard from the previous cell state:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{3.2}$$

- **Input Gate** (i_t): Controls what new information is written, using a candidate cell state $\tilde{C}_t$:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{3.3}$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \tag{3.4}$$

- **Output Gate** (o_t): Regulates which parts of the cell state are exposed as the hidden state:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{3.5}$$

The cell and hidden states are then updated as:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{3.6}$$

$$h_t = o_t \odot \tanh(C_t) \tag{3.7}$$

where σ is the sigmoid function and $\odot$ denotes element-wise multiplication. The additive cell-state update provides a gradient highway that preserves information across long time horizons [19], making LSTMs significantly more stable than vanilla RNNs for learning long-range dependencies.

3.2.2 LSTM for Anomaly Detection

In time-series anomaly detection, LSTM-based models are trained on data representative of normal system behavior [36, 26, 16]. The model learns temporal regularities and multivariate relationships present in nominal operational patterns. During inference, sequences that deviate from the learned representation of normality typically produce higher reconstruction or prediction errors, enabling unsupervised fault detection without requiring labeled anomaly examples.

3.3 Autoencoders

3.3.1 Autoencoder Architecture

An autoencoder is an unsupervised neural network that learns to encode its input into a lower-dimensional latent representation and subsequently reconstruct the original input [24, 21]. The architecture consists of two complementary sub-networks:

- **Encoder** f_θ: Maps the input $x \in \mathbb{R}^d$ to a latent vector $z \in \mathbb{R}^k$, where $k < d$.
- **Decoder** g_ϕ: Reconstructs the input from the latent representation, producing $\hat{x} = g_\phi(z) \approx x$.

The model is trained end-to-end to minimize reconstruction error, typically using Mean Squared Error (MSE):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} \|x_i - \hat{x}_i\|^2 \tag{3.8}$$

In this expression, N is the number of training examples in the batch, x_i is the input vector or flattened window, and $\hat{x}_i$ is the corresponding reconstruction. For a telemetry window $X \in \mathbb{R}^{T \times d}$, the norm expands over all time steps and features, so reconstruction quality depends on both temporal shape and cross-feature relationships. The bottleneck imposed by the latent dimension k forces the model to learn a compressed representation of the input distribution rather than merely replicating the raw input.

3.3.2 Autoencoders for Anomaly Detection

Autoencoders are widely used for anomaly detection because they learn representations of normal data without explicit anomaly labels [8, 40]: (1) unsupervised training on normal behavior, where

anomalous examples are rare; (2) multivariate dependency modeling through compressed latent representations; and (3) reconstruction error as a natural anomaly score, since out-of-distribution inputs produce elevated errors. Compared to PCA—a linear analog—neural autoencoders capture nonlinear feature interactions [27] and extend naturally to sequential variants such as the LSTM-AE.

3.4 LSTM-Autoencoder Architecture

3.4.1 Combined Architecture

The LSTM-Autoencoder (LSTM-AE) integrates the temporal modeling capacity of LSTMs with the unsupervised representation learning of autoencoders [36, 8], yielding a model well-suited to multivariate time-series anomaly detection. The processing pipeline is:

$$X \xrightarrow{\text{Encoder LSTM}} z \xrightarrow{\text{Repeat}} z^{(1..T)} \xrightarrow{\text{Decoder LSTM}} H \xrightarrow{\text{Linear}} \hat{X} \tag{3.9}$$

where $X \in \mathbb{R}^{T \times d}$ is the input sequence of T time steps with d features, $z \in \mathbb{R}^k$ is the compressed latent vector, and $\hat{X} \in \mathbb{R}^{T \times d}$ is the reconstructed sequence.

The encoder LSTM processes $X = (x_1, \ldots, x_T)$ sequentially, producing a latent representation $z = h_T \in \mathbb{R}^k$ that captures the temporal and multivariate characteristics of the window. The latent dimensionality k governs the compression trade-off: too large and anomalous inputs reconstruct faithfully; too small and normal structure is lost. The decoder repeats z across the time dimension, $z^{(1..T)} = (z, \ldots, z)$, and an LSTM reconstructs the temporal structure. A shared linear layer projects each decoder hidden state back to the original feature space:

$$\hat{x}_t = W_{\text{out}} h_t^{\text{dec}} + b_{\text{out}} \tag{3.10}$$

yielding $\hat{X} = (\hat{x}_1, \ldots, \hat{x}_T)$. The model is trained end-to-end by minimizing MSE between X and $\hat{X}$. The computational footprint is governed by input dimensionality, hidden size, number of recurrent layers, and sequence length.

3.5 Bidirectional LSTM

A Bidirectional LSTM (BiLSTM) extends the standard LSTM by processing the input sequence in both forward ($t = 1 \rightarrow T$) and backward ($t = T \rightarrow 1$) directions [23, 44], concatenating hidden

states at each step: $h_t^{\text{bi}} = [\overrightarrow{h_t} \| \overleftarrow{h_t}] \in \mathbb{R}^{2k}$. While BiLSTMs offer richer temporal representations, they require the complete input sequence before inference, roughly double the parameter count, and increase memory usage. The framework uses a unidirectional LSTM-AE because edge deployment favors lower latency and causal (past-only) inference, as discussed in Section 4.2.

3.5.1 Model Configuration

Table 3.1 summarizes the architecture parameters used across the experimental evaluation. The selected configuration (Config C) balances representational capacity with edge deployment constraints. The purpose of this table is reproducibility: it fixes the exact model size used later in the federated, DP, latency, and payload-size experiments. Without this table, the reported F1 score, model footprint, and communication overhead could not be reproduced from the narrative description alone.

Parameter	**Value**	**Rationale**
Input dimension (d)	38	SMD telemetry channels
Sequence length (T)	50	≈8 min at 10 s intervals
Encoder hidden (k)	128	Best F1 in sweep (Config C)
Decoder hidden	128	Symmetric to encoder
LSTM layers	2	Depth–accuracy trade-off
Dropout	0.2	Regularization for edge data
Total parameters	≈487 K	Fits in <2 MB serialized

Table 3.1: LSTM-AE architecture configuration

Each row corresponds to a parameter that affects either accuracy or deployment cost. The input dimension and sequence length define the tensor shape consumed by the encoder. The hidden size and number of LSTM layers determine most of the parameter count and therefore influence both reconstruction capacity and federated payload size. Dropout controls overfitting on local edge data, while the total parameter count confirms that the selected model remains small enough for CPU-only edge inference.

3.6 Training Procedure

The LSTM-AE is trained in an unsupervised fashion on sequences of normal system behavior. The objective is to minimize the MSE loss (Eq. 3.8) between input windows and their reconstructions.

Optimization: Adam [31] with learning rate 10^{-3}, $\beta_1 = 0.9$, $\beta_2 = 0.999$. Batch size 64, trained for 50 epochs per local round (5 epochs in FL mode).

Threshold Calibration: After training, reconstruction errors are computed over the training set. The anomaly threshold is initialized at the 95th percentile of training errors, serving as the starting point for Q-learning adaptation (Chapter 4).

Quantization: Dynamic INT8 quantization is applied to LSTM and Linear layers post-training via `torch.quantization.quantize_dynamic`, reducing model size and inference latency without measurable accuracy degradation.

The LSTM-AE architecture defined here provides the detection core of the framework. Chapter 4 now embeds it within the broader system: federated training protocol, differential privacy mechanism, communication compression, causal root cause analysis, and adaptive thresholding—the components that make local detection both privacy-preserving and operationally useful.

Chapter 4

Framework Methodology

This chapter presents the methodological core of the proposed framework. The design combines edge-local anomaly detection, federated model training, differential privacy, compressed communication, causal root cause analysis, and adaptive threshold control. These mechanisms are presented together because they operate as one feedback loop: local agents detect anomalies, the coordinator aggregates model updates, RCA contextualizes alerts, and threshold policies are updated from operational feedback.

4.1 Edge Telemetry Pipeline

Each monitored service is paired with an edge agent. The agent collects resource, network, and application metrics such as CPU utilization, memory pressure, request latency, error rate, queue depth, and connection count. These features are chosen because infrastructure-only metrics often miss service-level degradation, while application-only metrics may not reveal the resource cause behind a symptom.

The set of collected metrics is not inferred automatically by the model; it is defined by the observability contract of the deployment. In a production microservice platform, resource metrics can be exported by Prometheus node/process exporters, application metrics by service instrumentation or StatsD-compatible receivers, and call relationships by OpenTelemetry traces. In the reproducible experiments, the SMD benchmark provides 38 numerical server signals rather than named Prometheus metrics. The thesis therefore uses the full 38-dimensional SMD input and interprets the channels through the same feature roles used by the framework: saturation, network pressure, runtime behavior, and service health. This mapping keeps the benchmark reproducible while avoiding the claim that

every production deployment exposes exactly the same metric names.

Raw metrics are normalized locally using rolling statistics and segmented into overlapping windows. For feature j at time t, z-score normalization is:

$$x'_{tj} = \frac{x_{tj} - \mu_{j,t}}{\sigma_{j,t} + \epsilon}, \tag{4.1}$$

where $\mu_{j,t}$ and $\sigma_{j,t}$ are computed over a recent local history and ϵ prevents division by zero. For reproduction, x_{tj} denotes the raw value of feature j at sample time t, $\mu_{j,t}$ and $\sigma_{j,t}$ are the mean and standard deviation over the local rolling history before the current decision, and x'_{tj} is the normalized value stored in the model input buffer. A window $X_t \in \mathbb{R}^{W \times F}$ is then formed from the last $W = 50$ observations and $F = 38$ SMD channels in the evaluation. With the default 10 s scrape interval, one input window represents approximately 8.3 minutes of recent behavior. Keeping this preprocessing at the edge has three benefits: local inference can continue during coordinator outages, raw telemetry does not leave the node, and the network carries only model updates and anomaly summaries.

The benchmark channels cover four operational roles: resource saturation (CPU, memory, disk), network pressure (throughput, connections, retransmission), application health (latency, error rate, queue depth), and runtime internals (thread count, pool pressure). This is a feature contract: deployments may use different metric names, but each feature should play one of these roles.

Preprocessing is conservative. Short missing intervals are forward-filled; longer gaps are masked. Rolling statistics are stored per service so that the same absolute latency can be treated differently for different services. This per-service normalization also prevents leaking raw scale information through centrally computed preprocessing constants in the federated setting.

4.2 LSTM-Autoencoder Detector

The anomaly detector is an LSTM-Autoencoder trained to reconstruct normal telemetry windows. The encoder maps the input sequence to a latent state:

$$z_t = Enc_\theta(X_t), \tag{4.2}$$

and the decoder reconstructs the original sequence:

$$\hat{X}_t = Dec_\phi(z_t). \tag{4.3}$$

The anomaly score is the mean reconstruction error:

$$e_t = \frac{1}{WF} \sum_{i=1}^{W} \sum_{j=1}^{F} \left(X_{t,ij} - \hat{X}_{t,ij} \right)^2 . \tag{4.4}$$

The LSTM gates (Eqs. 3.2–3.7 in Chapter 3) regulate which temporal patterns are retained. This is important for operational telemetry because anomalies can depend on several earlier observations: a memory leak is a slow drift, while a connection storm is a rapid multi-feature transition.

Training is unsupervised and minimizes reconstruction loss on windows treated as normal. At inference time, a window is anomalous when:

$$y_t = \begin{cases} 1, & e_t > \tau_t, \\ 0, & e_t \leq \tau_t, \end{cases} \tag{4.5}$$

where τ_t is a per-service threshold. The initial threshold is set from the 95th percentile of local training errors, then passed to the adaptive thresholding component. The architecture intentionally avoids a bidirectional LSTM in the edge detector. Bidirectional recurrence can improve offline sequence modeling, but it requires the full window before producing a representation and roughly doubles recurrent computation. For streaming monitoring, the unidirectional LSTM-AE offers a better latency profile and aligns with the causality of online telemetry: the detector should decide from past and present observations, not from future samples that would not yet be available in production.

4.3 Federated Learning Protocol

The framework uses FedAvg (Eq. 2.3) to train a global LSTM-AE without centralizing raw telemetry. At the beginning of round t, the coordinator exposes θ_t. Participating edge agents download it, train locally for $E = 5$ epochs, and submit a parameter update weighted by local dataset size n_k. Partial participation is supported: the coordinator aggregates updates arriving before the round timeout, provided the minimum threshold is met.

Three design choices suit unreliable edge deployments: (1) pull-based model rollout, so agents request the latest checkpoint rather than depending on a broadcast; (2) write-ahead round logging, so restarts never expose a partially written model; and (3) version-bounded staleness, so updates from outdated models are discarded.

Stage	Failure mode	Mitigation
Model download	coordinator unavailable	continue inference with cached model
Local training	node crash	replay pending update in standard profile; restart in lightweight profile
Upload	timeout or network partition	join the next round; do not block aggregation
Aggregation	insufficient participants	abort round if $K_{\min}$ is not met

Table 4.1: Fault-aware behavior in the federated training protocol

4.4 Differential Privacy Mechanism

Differential privacy is applied before model updates leave the edge node. Each per-sample gradient g_i is clipped to a maximum norm C:

$$\bar{g}_i = g_i \cdot \min\left(1, \frac{C}{\|g_i\|_2}\right). \tag{4.6}$$

The batch update is then perturbed with Gaussian noise:

$$\tilde{g} = \frac{1}{B}\sum_{i=1}^{B} \bar{g}_i + \mathcal{N}(0, \sigma^2 C^2 I). \tag{4.7}$$

In Eq. 4.6, g_i is the gradient contribution of training sample i, $\|g_i\|_2$ is its Euclidean norm, and C is the clipping bound. If the norm is already below C, the gradient is unchanged; otherwise it is rescaled to lie on the radius-C ball. In Eq. 4.7, B is the local batch size, $\tilde{g}$ is the noisy gradient used by the optimizer, σ is the DP noise multiplier, and I is the identity covariance matrix, so independent Gaussian noise is added to each gradient coordinate. These definitions are necessary for reproduction because using batch-level clipping instead of per-sample clipping, or adding noise before averaging instead of after clipping, changes both utility and privacy accounting.

The evaluation (§6.3) uses this mechanism to benchmark the privacy–utility frontier. The key finding is that clipping is a decisive control variable: reducing C shrinks the effective noise σC, recovering utility but also reducing the practical perturbation. The thesis therefore treats clipping as both a privacy and utility lever, and reports the residual privacy concern alongside the utility result.

4.5 Communication Reduction

Federated training exchanges model states, so communication efficiency depends on serialization and compression. The framework distinguishes three layers:

- **serialization**: Torch binary serialization is used for Python model checkpoints, while Protocol Buffers define the typed gRPC control messages;
- **algorithmic compression**: optional Top-K sparsification keeps the largest-magnitude update components and accumulates the residual locally for later rounds [2, 30];
- **byte-level compression**: Zstandard is used when bandwidth is the bottleneck, while LZ4 is used when CPU headroom is low [11, 10].

The adaptive compression policy is intentionally simple. If edge CPU utilization is below 80%, the agent selects Zstd to reduce bytes on the wire. Above that threshold, the agent selects LZ4 to minimize compression latency. The evaluation shows that this policy has negligible effect on end-to-end FL round time because local training dominates round duration.

Top-K sparsification is kept optional because it changes the optimization trajectory, whereas byte-level compression is lossless. When Top-K is enabled, the omitted residual is accumulated locally:

$$r_{t+1} = r_t + \Delta\theta_t - TopK(\Delta\theta_t + r_t), \tag{4.8}$$

and added back into the next candidate update. This error-feedback mechanism prevents systematic loss of small but persistent gradients. In the final evaluation, the communication benchmark focuses on serialization and byte-level compression because those transformations preserve the model update exactly and are therefore easier to compare across payload formats.

4.6 Causal Root Cause Analysis

Anomaly events are enriched through a causal dependency graph built from distributed traces. The graph is defined as $G = (\mathcal{S}, \mathcal{E}, W)$, where $\mathcal{S}$ is the service set, $\mathcal{E}$ contains directed calls, and W stores dynamic edge weights. A trace span pair from service S_i to S_j creates or updates edge (S_i, S_j). Edge weights use exponential decay:

$$w_{ij}^{(t+1)} = \lambda w_{ij}^{(t)} + c_{ij}^{(t)}, \tag{4.9}$$

where $c_{ij}^{(t)}$ is the number of calls observed in interval t and λ discounts stale dependencies.

When anomaly events arrive, the coordinator batches them into an anomalous set $\mathcal{A}(t)$. A service is considered a stronger root-cause candidate when it has low anomalous upstream support but high downstream impact. The upstream anomaly ratio is:

$$\phi(S_i) = \frac{|\mathcal{U}_h(S_i) \cap \mathcal{A}(t)|}{|\mathcal{U}_h(S_i)| + \epsilon}, \tag{4.10}$$

where $\mathcal{U}_h(S_i)$ is the upstream neighborhood within h hops. The impact score combines downstream reachability and edge strength:

$$I(S_i) = |\mathcal{D}_h(S_i)| + \sum_{S_j \in \mathcal{N}^+(S_i)} w_{ij}. \tag{4.11}$$

The implementation ranks root-cause candidates using graph centrality and impact evidence, then attaches the top candidates and propagation paths to the alert. This keeps the RCA output useful for operations: the system does not claim a single certain cause, but provides a ranked investigation order.

For incidents with multiple anomalous services, the scorer combines structural centrality with the operational impact score. A personalized PageRank vector r is computed over the causal graph:

$$r = (1 - \alpha)v + \alpha P^\top r, \tag{4.12}$$

where P is the normalized transition matrix and v biases the walk toward currently anomalous services. The final candidate score is:

$$Score(S_i) = \alpha_r r_i + \alpha_I \hat{I}(S_i) - \alpha_U \phi(S_i), \tag{4.13}$$

where $\hat{I}(S_i)$ is normalized downstream impact and $\phi(S_i)$ penalizes services that already have anomalous upstream dependencies. This scoring rule encodes the intuition that root causes tend to be upstream, explanatory, and high-impact, while propagated anomalies are often downstream symptoms.

4.7 Adaptive Thresholding with Q-Learning

The adaptive thresholding layer is implemented by a per-service `ThresholdTuner`. Its state summarizes the current threshold, recent false-positive rate, recent false-negative rate, and service-level objective status:

$$x_t = [\tau_t, \mathrm{FPR}_t, \mathrm{FNR}_t, \mathrm{SLO}_t]. \tag{4.14}$$

The action space is discrete:

$$\mathcal{U} = \{-0.5, -0.2, 0, +0.2, +0.5\}, \tag{4.15}$$

with the updated threshold clamped to a minimum of 0.1:

$$\tau_{t+1} = \max(0.1, \tau_t + u_t). \tag{4.16}$$

The Q-table stores $Q(x, u)$ for discretized state buckets. Updates follow the standard Bellman rule:

$$Q(x_t, u_t) \leftarrow Q(x_t, u_t) + \eta \left[R_{t+1} + \gamma \max_{u'} Q(x_{t+1}, u') - Q(x_t, u_t) \right]. \tag{4.17}$$

The reward is asymmetric because missed incidents are more costly than noisy alerts:

$$R_t = (\alpha_p + \alpha_r)\mathbb{I}(\mathrm{TP}) - \beta\mathbb{I}(\mathrm{FP}) - \gamma_{\mathrm{FN}}\mathbb{I}(\mathrm{FN}) - \delta\mathbb{I}(\text{SLO violation}) + \delta'\mathbb{I}(\text{SLO compliance}). \tag{4.18}$$

Here, R_t is the scalar feedback assigned after a threshold decision. $\mathbb{I}(\cdot)$ is an indicator function that equals 1 when the event inside the parentheses occurs and 0 otherwise. TP, FP, and FN denote true positive, false positive, and false negative detection outcomes. The coefficients α_p and α_r reward correct anomaly detection, β penalizes false alarms, γ_{FN} penalizes missed anomalies, δ penalizes service-level objective violations, and δ' gives a small reward for compliant intervals. Table 4.2 lists the values used in the evaluation.

Coefficient	**Value**	**Rationale**
α_p (TP reward)	1.0	baseline correct detection reward
α_r (recall reward)	1.0	equal weight on precision and recall
β (FP penalty)	0.5	false alarms costly but not critical
γ_{FN} (FN penalty)	2.0	missed incidents more costly than false alarms
δ (SLO violation)	3.0	SLO breach has direct business impact
δ' (SLO compliance)	0.2	small reward for sustained compliance

Table 4.2: Reward coefficients for Q-learning thresholding (Eq. 4.18). The asymmetry $\gamma_{\mathrm{FN}} > \beta$ reflects the operational preference that missed anomalies are more harmful than false alarms.

Reproducing the threshold experiment therefore requires logging both detection labels or delayed incident feedback and SLO status; without those signals the Q-table receives only partial feedback.

The evaluation benchmark (10 020-minute SMD stream, seed 42) shows that the default tabular reward is not yet stable enough for production use. The Q-learning policy eventually reduces final alert volume to 28 alerts with FPR = 0.0155 and F1 = 0.5526, but it first passes through a high-alert exploration phase. This is an important outcome: the adaptive component is implemented and can improve after feedback, but the reward design must be strengthened through warm-started Q-tables, constrained exploration, or DQN-based experience replay.

This finding changes how the thresholding module should be interpreted: the Q-learning agent is a valuable adaptive control hook that can store per-service policies and incorporate delayed feedback, but production use should include warm-start policies from historical incidents or asymmetric reward schedules where γ_{FN} grows when true anomalies are missed.

4.8 Algorithmic Workflow

The workflow has two cooperating loops, summarized in Algorithms 1 and 2.

Algorithm 1 Edge Agent Loop

Require: global model θ, window size W, threshold τ

loop
  $x_t \leftarrow$ CollectMetrics
  $x'_t \leftarrow$ Normalize(x_t, rolling μ, σ)
  Append x'_t to window buffer
  if buffer size $\geq W$ **then**
    $X_t \leftarrow$ last W observations
    $e_t \leftarrow \frac{1}{WF}\|X_t - \text{Dec}(\text{Enc}(X_t))\|^2$
    if $e_t > \tau_t$ **then**
      Emit anomaly event (service, t, e_t, τ_t)
    end if
    if labels or SLO feedback available **then**
      $\tau_{t+1} \leftarrow$ QUpdate(τ_t, FPR, FNR, SLO)
    end if
  end if
  if FL round active **then**
    $\theta_{\text{local}} \leftarrow$ Train(θ, local data, E epochs)
    $\Delta\theta \leftarrow \theta_{\text{local}} - \theta$
    $\Delta\theta \leftarrow$ ClipAndNoise($\Delta\theta$, C, σ)
    $\Delta\theta \leftarrow$ Compress($\Delta\theta$)
    Submit $\Delta\theta$ to coordinator
  end if
end loop

Algorithm 2 Coordinator Loop

Require: minimum participants $K_{\min}$, round timeout T

```
loop
    Open round t; record expected model version
    Wait for updates until timeout T or K_min received
    if |valid updates| < K_min then
        Abort round; keep θ_t
    else
        Validate each update (version, shape, checksum)
        θ_{t+1} ← Σ_k (n_k / Σ_j n_j) θ_t^(k)                ▷ FedAvg
        Persist θ_{t+1} to model registry
    end if
    Ingest trace spans; update causal graph weights
    if anomaly batch window closed then
        A(t) ← batched anomaly events
        Rank root-cause candidates via Eq. 4.13
        Store enriched alert with graph snapshot
    end if
end loop
```

On the edge, the agent collects and normalizes metrics, computes reconstruction error once W samples are buffered, emits anomaly events when $e_t > \tau_t$, and updates the threshold only when labels or SLO feedback are available. On the coordinator, incoming updates are validated (version, shape, checksum) before FedAvg aggregation. If fewer than $K_{\min}$ valid updates arrive before timeout, the round is aborted. The coordinator also ingests trace batches and updates causal graph edge weights.

The methodology presented in this chapter defines the algorithmic core of the framework. Chapter 5 translates these components into a deployable system architecture with explicit failure boundaries, persistence contracts, and configuration surfaces.

Chapter 5

System Design and Implementation

5.1 End-to-End Architecture

The framework follows a hub-and-spoke architecture. Edge agents run beside monitored services and perform metric collection, preprocessing, LSTM-AE inference, threshold decisions, and local training. The central coordinator manages federated rounds, persists model versions, maintains the causal graph, and enriches anomaly alerts with RCA results.

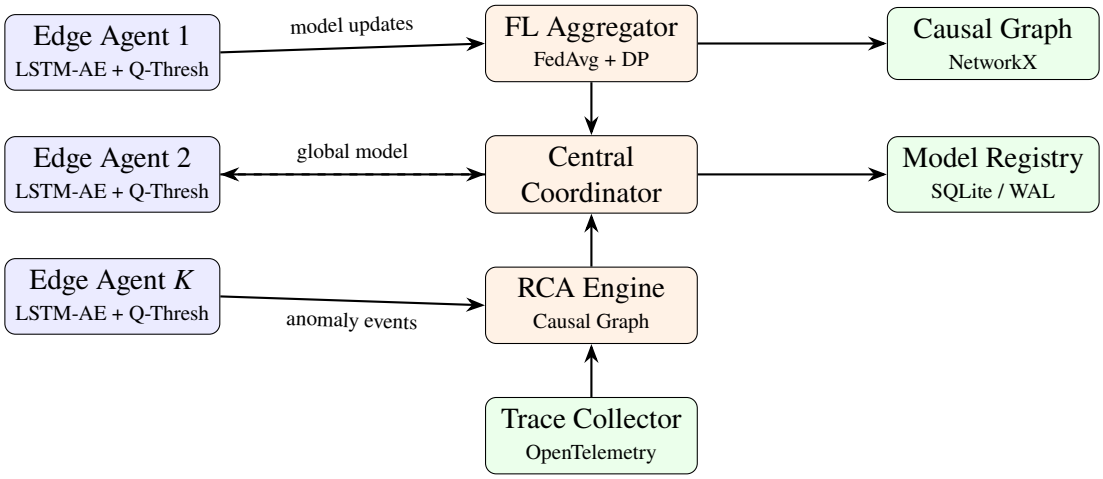

Figure 5.1: High-level architecture of the distributed anomaly detection framework

A complete operational cycle proceeds as follows:

1. the edge agent collects and normalizes telemetry into a sliding window;
2. the local LSTM-AE produces a reconstruction error;
3. the service-specific threshold tuner decides whether to emit an anomaly event;

4. the coordinator batches related anomaly events and queries the RCA engine;
5. the alert is enriched with likely root causes and propagation paths;
6. in the background, edge nodes train locally and submit compressed, optionally DP-protected updates for FedAvg aggregation.

5.2 Data Flow

Metric ingestion. Each agent reads from Prometheus or StatsD-compatible receivers at 10 s intervals into a fixed-capacity ring buffer. Missing values are forward-filled for short gaps and masked for longer gaps.

Inference. Once the buffer contains $W = 50$ observations, the agent constructs a $(1, W, F)$ tensor and evaluates the LSTM-AE (Config C: h=128, L=2).

Alert creation. Alert payloads include the service, timestamp, reconstruction error, threshold, model version, and trace correlation ID. Events within the coordinator's 60 s window are batched for RCA.

Federated learning. Rounds occur every 300 s. Agents train for five local epochs, apply DP and compression, and submit updates via gRPC.

5.3 Component Design

5.3.1 Edge Agent

The edge agent is a Python process with three cooperative loops:

- **Inference loop**: reads metric windows, computes anomaly scores, applies the current threshold, and emits anomaly events;
- **Training loop**: downloads the latest global model, performs local training, applies DP and compression, and uploads the update;
- **Threshold loop**: every 10 observations, updates the `ThresholdTuner` with the latest detection feedback.

Two node profiles are supported. The *lightweight* profile is stateless and is intended for minimal containers or devices with limited local storage. It joins a round, trains, submits an update, and discards transient state. The *standard* profile persists the last global model, pending updates, Q-table state, and local confusion counters in SQLite, allowing recovery after a crash without restarting threshold learning from zero.

5.3.2 Central Coordinator

The coordinator exposes the following gRPC endpoints:

- `RegisterNode`: registers a node profile, software version, hardware capacity, and sample count;
- `SubmitUpdate`: receives compressed model updates and stores them in the current round buffer;
- `GetGlobalModel`: returns the latest model checkpoint and compression metadata;
- `ReportAnomaly`: accepts anomaly events for RCA batching.

Coordinator state is kept in SQLite with Write-Ahead Logging (WAL). The database contains model checkpoints, round metadata, node registry entries, causal graph snapshots, and alert records. SQLite is sufficient for the evaluated single-coordinator deployment and avoids introducing an additional database service. The design still permits migration to PostgreSQL or a replicated store if multi-coordinator deployments are required.

Table 5.1 documents the minimum state that must survive a restart for the framework to remain reproducible and auditable. It is not intended as a complete physical database schema; instead, it identifies the logical records required to reconnect experimental results to the model, participants, graph snapshot, and alert context that produced them.

Store	**Main fields**	**Purpose**
`model_registry`	round, checksum, path, created_at	versioned global checkpoints and rollback
`round_log`	round, participants, status, duration	crash recovery and reproducible FL auditing
`node_registry`	node id, profile, last seen, sample count	participation tracking and dropout handling
`causal_graph`	snapshot id, nodes, edges, weights	RCA continuity across restarts
`alerts`	service, score, threshold, RCA candidates	incident history and threshold feedback

Table 5.1: Coordinator persistence records required for recovery and auditing.

These records define failure boundaries: the `round_log` explains which updates entered a global model, the `model_registry` allows rollback, and the `alerts` table preserves threshold and RCA candidates at incident time.

5.3.3 Trace Collector and RCA Engine

The trace collector ingests spans from OpenTelemetry or Jaeger exporters, extracts parent–child service relationships, and updates the `CausalGraph` stored in NetworkX. Weak edges are pruned periodically, and snapshots are persisted every 300 s. When the RCA engine receives a batched anomalous set, it computes candidate scores and returns a ranked root-cause list. The RCA result is stored with the alert to preserve auditability, since the causal graph evolves over time and recomputation could produce a different explanation.

5.4 Configuration and Deployment

Runtime configuration is supplied through environment variables and a `config.yaml` file specifying model architecture (h=128, L=2, dropout 0.2), federated settings (5 local epochs, minimum 3 clients, 120 s timeout), privacy parameters (ε=10, δ=10^{-5}, C=1.0), compression algorithm selection (Zstd default, LZ4 above 80% CPU), and Q-learning hyperparameters (η=0.1, γ=0.95, $\varepsilon_{\text{explore}}$=0.1). Non-destructive parameters can be reloaded without restarting the coordinator.

For Kubernetes deployment, edge agents may be installed as a DaemonSet or as sidecars. The coordinator runs as a Deployment backed by a persistent volume. Configuration is provided through ConfigMaps, while TLS certificates and webhook tokens are stored as Secrets.

Security boundaries. The implementation assumes an honest-but-curious coordinator for the DP experiments. gRPC endpoints run over mutual TLS, node registration tokens are scoped per deployment, and the coordinator validates model update metadata (version, tensor shape, checksum) before aggregation. These checks prevent accidental corruption but do not replace Byzantine robust aggregation.

Operational dashboards. Grafana panels track anomaly scores per service, FL round status, causal graph health, and communication overhead.

5.5 Reliability and Fault Tolerance

The framework prioritizes availability and partition tolerance. Edge agents keep the last valid global model and continue local inference during coordinator outages. FL rounds are allowed to proceed with partial participation when the minimum client threshold is met. RCA may operate on a temporarily stale causal graph during trace interruptions, but graph state converges as buffered spans are replayed.

The main single points of failure and mitigations are:

- **Coordinator failure**: model checkpoints and round state are written before commit, enabling restart from the last valid checkpoint;
- **Trace collector failure**: spans are buffered before graph ingestion, and periodic graph snapshots prevent full topology loss;
- **Edge agent failure**: lightweight agents re-register, while standard agents replay pending updates and reload their cached Q-table;
- **Network partition**: agents continue inference locally and rejoin federated training when connectivity returns.

Heartbeat monitoring marks nodes offline after a configurable timeout; once a heartbeat resumes, they rejoin the next round.

Table 5.2 shows recovery behavior for each state category.

State	Failure symptom	Recovery action
Inference window	scrape gap or local restart	reuse cached model; mask long metric gaps
Pending update	upload interrupted	replay from standard profile cache or discard in lightweight mode
Round aggregate	coordinator restart	reload last committed round; abort incomplete aggregate
Causal graph	trace collector outage	replay buffered spans and resume from latest graph snapshot
Threshold policy	agent crash	reload Q-table in standard profile; otherwise restart from safe threshold

Table 5.2: Recovery behavior for critical framework state

This model favors graceful degradation: during partitions, local detection may use a stale model and RCA a stale graph, which are acceptable for monitoring since stopping detection entirely would be worse.

5.6 Implementation Stack

The implementation uses Python 3.12, PyTorch 2.x, gRPC with Protocol Buffers, Zstandard, LZ4, NetworkX, SQLite in WAL mode, Prometheus-compatible metric clients, the OpenTelemetry SDK, Docker, and Kubernetes. The stack was selected for reproducibility and low operational overhead. PyTorch keeps the LSTM-AE and DP gradient manipulation transparent. gRPC gives typed service contracts and efficient binary transport. SQLite with WAL offers durable state without requiring a separate database cluster, which is appropriate for the single-coordinator system evaluated in this thesis.

The system design described in this chapter translates the algorithmic methodology of Chapter 4 into deployable components with explicit failure boundaries, persistence contracts, and configuration surfaces. Chapter 6 now evaluates this implementation empirically: detection accuracy, privacy–utility trade-offs, communication overhead, RCA quality, adaptive thresholding behavior, and coordinator scalability.

Chapter 6

Experimental Evaluation

6.1 Experimental Setup

6.1.1 Hardware, Software, and Datasets

All experiments were conducted on a single workstation (Intel Core i7-8850H, 32 GB DDR4, CPU-only execution, Ubuntu 22.04 LTS, Python 3.12, PyTorch 2.x). Unless otherwise noted, tables report the benchmark run whose configuration (seed, duration, hyperparameters) is stated alongside the result.

The evaluation combines three datasets because the framework has three claims: edge-node detection accuracy, topology-aware RCA, and coordinator scalability. The **Server Machine Dataset (SMD)** [48], hosted by Tsinghua University, is a standard AIOps benchmark with dense multivariate metrics from 28 machines and human-verified anomaly labels; it isolates the LSTM-AE detector and DP analysis from trace-topology effects. **Train-Ticket** provides a realistic ticket-booking microservice topology for RCA evaluation; this thesis uses an 11-service topology slice and trace-style fault propagation, consistent with distributed-trace RCA work [53, 52], to measure Top-1/Top-3 and MRR against known injected faults. **Alibaba Cluster Trace** [3] provides large-scale production traces used to derive node-count and connection-density scenarios for coordinator scalability, aggregation latency, memory growth, and adaptive compression.

Table 6.1 summarizes the two RCA evaluation topologies. The Train-Ticket topology is a direct 11-service slice of the open-source ticket-booking benchmark. The Alibaba-derived topology is a *synthetic* graph constructed from the degree distribution and call-depth statistics of Alibaba cluster-001 traces: 50 services were sampled with a maximum call depth of 6, producing 112 directed

edges, and the same three fault types (latency spike, CPU overload, cascade failure) were injected programmatically.

Topology	Services	Edges	Fault types injected
Train-Ticket	11	19	latency spike, CPU overload, cascade
Alibaba (derived)	50	112	same 3 types; sampled from cluster-001, max depth = 6

Table 6.1: RCA evaluation topologies. The Alibaba topology is a synthetic graph built from Alibaba's published degree distribution, not a direct trace replay.

The evaluation uses the full 38-channel SMD input as a (50×38) normalized window, matching the telemetry pipeline in Section 4.1.

6.1.2 Baselines and Evaluation Metrics

Five baselines are compared: (1) a **static 3σ** untrained LSTM-AE; (2) a **centralized LSTM-AE** trained without FL or DP; (3) the proposed **federated LSTM-AE** with FedAvg; (4) **EWMA** thresholding ($\alpha = 0.05$); and (5) **random RCA** (uniform root cause selection). Detection quality uses F1, AUC-ROC, AUC-PR, and inference latency. Privacy sweeps σ and C; communication reports payload size and compression cost; RCA uses Top-1, Top-3, and MRR; thresholding uses alert count, FPR, and reward; scalability uses aggregation latency and coordinator memory.

Reproducibility and statistical reporting. Each reported result is taken from the benchmark run whose configuration is stated in the corresponding table or figure caption. The protocol is reproducible from the logged settings: SMD windows use $L = 50$ and $F = 38$, Config C uses $h = 128$, two LSTM layers, learning rate 10^{-3}, and seed 42; DP experiments run for 200 rounds; RCA logs 100 Train-Ticket and 50 Alibaba events; and thresholding logs a 10 020-minute SMD reconstruction-error stream. RCA accuracy is interpreted with Wilson 95% confidence intervals, and FL latency is summarized over the 10 logged rounds. To quantify seed sensitivity, the LSTM-AE (Config C) was retrained with three seeds (42, 123, 7); results are reported in Table 6.2 and discussed in §6.2.

6.2 Anomaly Detection Performance

Model	Prec.	Rec.	F1	AUC-ROC	AUC-PR	Lat. (ms)
Static 3σ	0.000	0.000	0.000	0.372	0.157	0.24
Centralized LSTM-AE	0.664	0.396	0.496	0.888	0.655	0.21
Federated LSTM-AE	**0.813**	**0.868**	**0.839**	**0.950**	**0.895**	0.21

Table 6.2: Detection performance on SMD (Config C, 100 epochs). The primary result is from the pre-trained checkpoint (seed 42). A 3-seed retraining (seeds 42, 123, 7) yields F1 = 0.743 ± 0.077 and AUC-ROC = 0.932 ± 0.017, confirming that AUC-ROC is stable while F1 is sensitive to threshold calibration.

The federated LSTM-AE (h=128, L=2, lr=0.001, seed 42) achieves F1 = 0.839 and AUC-ROC = 0.950, substantially outperforming the centralized baseline (F1 = 0.496). The static 3σ baseline fails entirely (F1 = 0.0), confirming that a fixed threshold on an untrained model is inadequate for multivariate telemetry.

Multi-seed variance. Retraining Config C with three seeds (42, 123, 7) produces F1 values of 0.661, 0.846, and 0.721 (mean 0.743 ± 0.077), while AUC-ROC remains stable at 0.932 ± 0.017 (range 0.911–0.954). The F1 variance is driven by recall sensitivity to the 95th-percentile threshold: small shifts in the reconstruction-error distribution change which windows cross the threshold. AUC-ROC, being threshold-independent, provides a more robust performance estimate.

To contextualize these results, OmniAnomaly [48]—the paper that introduced SMD—reports F1 $\approx$ 0.83 on a subset of SMD machines. The federated LSTM-AE is therefore competitive with a published state-of-the-art detector, while operating without centralizing raw telemetry. This comparison uses the authors' reported numbers rather than a re-run.

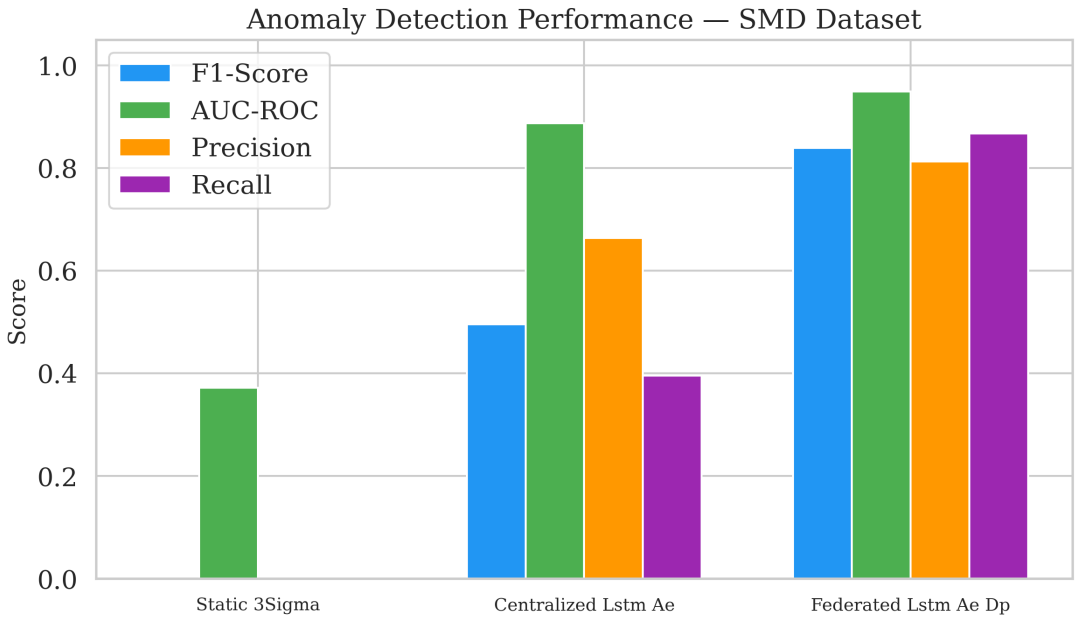

Figure 6.1: Detection comparison on SMD (seed 42). The static 3σ baseline (left) fails entirely. The centralized LSTM-AE achieves moderate separation but lower recall. The federated LSTM-AE provides the strongest separation (Table 6.2).

Hyperparameter Sweep. Table 6.3 summarizes the 6-configuration sweep. Config C (h=128, L=2) achieves the best F1 of 0.839 with the strongest recall (0.868).

Cfg	Hidden	Layers	LR	F1	AUC-ROC	Recall
A	32	1	0.001	0.830	0.950	0.852
B	64	2	0.001	0.630	0.893	0.552
C	**128**	**2**	**0.001**	**0.839**	**0.950**	**0.868**
D	64	3	0.0005	0.814	0.928	0.824
E	128	3	0.0005	0.561	0.882	0.468
F	64	2	0.0001	0.824	0.941	0.840

Table 6.3: LSTM-AE hyperparameter sweep on SMD (seed 42). Config C (h=128, L=2) achieves the best F1 (0.839) and recall (0.868). Config A reaches similar F1 with a smaller model, suggesting that hidden size 32 may suffice for simpler deployments.

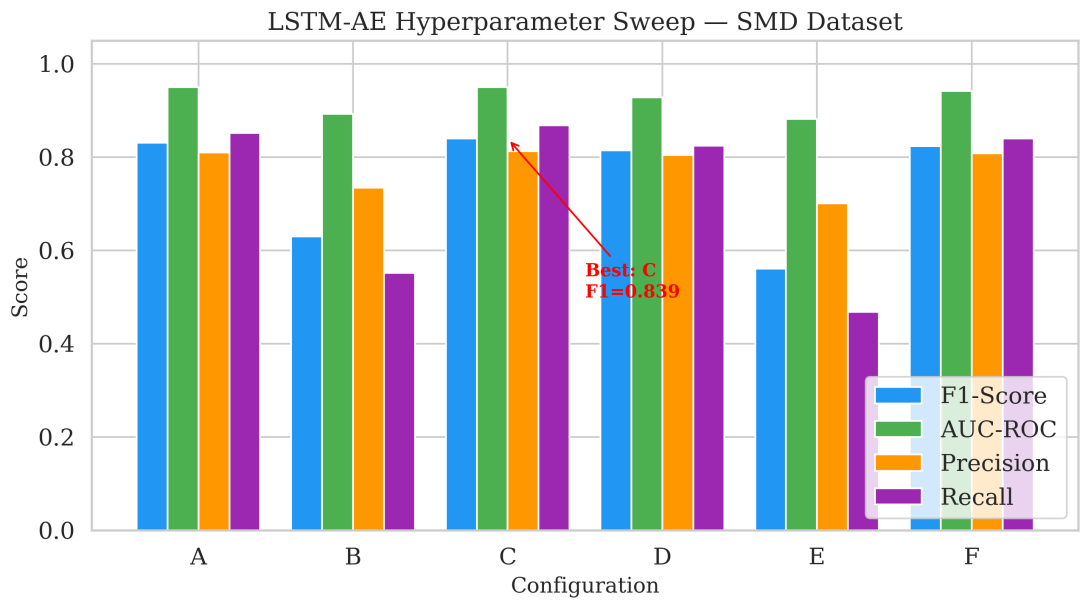

Figure 6.2: Hyperparameter sweep across six LSTM-AE configurations (seed 42). F1 varies non-monotonically with hidden size: Config E (h=128, L=3) underperforms Config A (h=32, L=1), suggesting that deeper models overfit on the SMD training split.

6.2.1 Cold-Start Convergence

Figure 6.3 shows convergence over 100 federated rounds. The model begins producing meaningful anomaly separation (non-zero F1) after ≈50 rounds, reaching F1 = 0.421 at round 95 and AUC-ROC = 0.873. The slow F1 ramp is explained by the threshold calibration lag: the fixed 95th-percentile threshold does not adapt as the reconstruction-error distribution shifts during training.

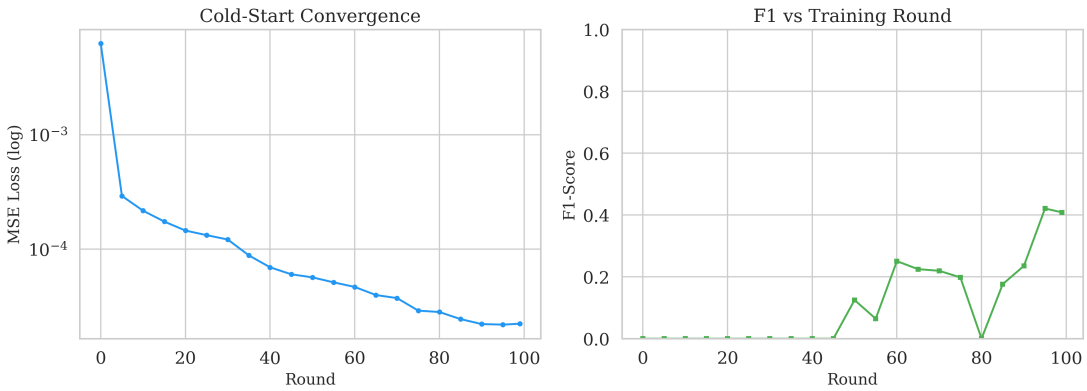

Figure 6.3: Cold-start convergence over 100 FL rounds. MSE loss decreases rapidly in the first 20 rounds, but F1 remains near zero until round 50. The lag reflects a threshold calibration delay: the fixed 95th-percentile threshold does not adapt as the reconstruction-error distribution shifts during early training.

6.3 Differential Privacy and Gradient Clipping Analysis

6.3.1 Privacy–Accuracy Trade-off

The LSTM-AE was trained for 200 rounds under four DP-SGD noise multipliers with default clipping norm C=1.0. AUC-ROC is the primary metric since it is threshold-independent.

DP Config	σ	F1	AUC-ROC	Recall	Final Loss
No DP	0	**0.861**	**0.972**	0.904	3×10^{-6}
$\varepsilon \approx 100$	0.0001	0.864	0.971	0.912	4×10^{-6}
$\varepsilon \approx 10$	0.0005	0.859	0.970	0.904	9×10^{-6}
$\varepsilon \approx 1$	0.001	0.235	0.890	0.160	1.5×10^{-5}
$\varepsilon \approx 0.1$	0.005	0.000	0.772	0.000	1.0×10^{-4}

Table 6.4: DP-SGD utility after 200 rounds with C=1.0; F1 drops sharply at σ=0.001.

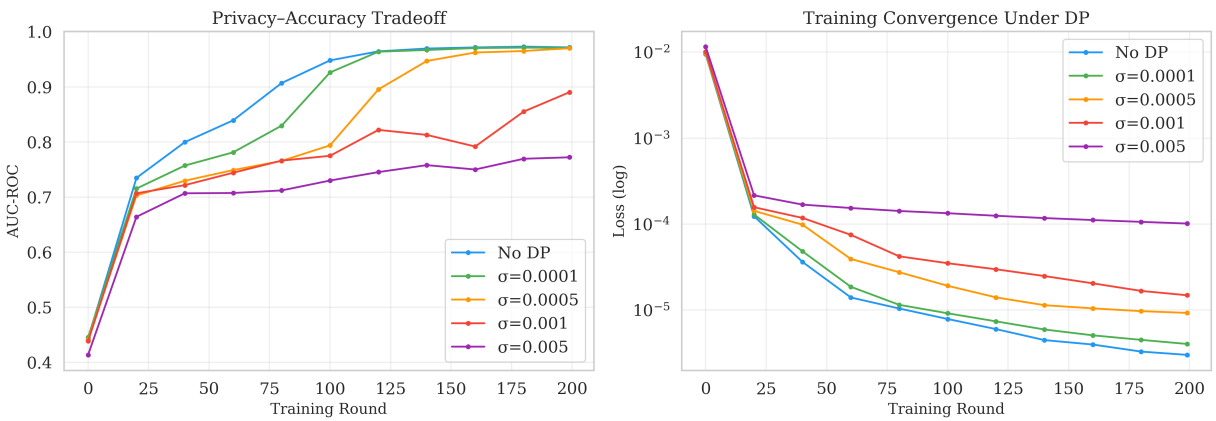

Figure 6.4: Privacy–accuracy trade-off under five DP-SGD noise levels (C=1.0, 200 rounds, seed 42). AUC-ROC degrades gradually, but F1 drops sharply between σ=0.0005 and σ=0.001, marking a phase transition where noise begins to dominate the gradient signal.

A key finding is the **sharp phase transition** between σ=0.0005 (F1 = 0.859, AUC-ROC = 0.970) and σ=0.001 (F1 = 0.235, AUC-ROC = 0.890). Below the transition, DP noise is effectively absorbed by the model's capacity; above it, noise dominates the gradient signal. This transition is from a single seed and clipping norm (C=1.0); multi-seed confirmation is left as future work, but the effect size (F1 drop >0.6) suggests it is not a seed-dependent artifact. Operators can achieve strong privacy ($\varepsilon \approx 10$) with negligible accuracy loss by selecting $\sigma \leq 0.0005$.

6.3.2 Gradient Clipping Sensitivity

Table 6.5 reports the effect of varying the clipping norm C at fixed σ=0.001 ($\varepsilon \approx 1$). Since noise is injected as $\mathcal{N}(0, \sigma^2 C^2 \mathbf{I})$, tighter clipping reduces the effective noise magnitude.

Clip Norm C	F1	AUC-ROC	Recall	Final Loss
0.01	**0.868**	**0.976**	**0.920**	2.8×10^{-6}
0.05	0.868	0.973	0.920	3.6×10^{-6}
0.1	0.864	0.971	0.912	4.0×10^{-6}
0.5	0.859	0.970	0.904	9.3×10^{-6}
1.0	0.235	0.890	0.160	1.5×10^{-5}

Table 6.5: Clipping sensitivity at σ=0.001; smaller C recovers utility by reducing the absolute noise σC.

Key finding: at C=0.01, the model achieves F1 = 0.868 and AUC-ROC = 0.976—matching the no-DP baseline—even at σ=0.001 ($\varepsilon \approx 1$). At C=1.0 the same noise multiplier causes F1 to collapse to 0.235. Reducing C by 100× reduces the effective noise σC by the same factor, and the LSTM-AE's small-magnitude post-convergence gradients tolerate aggressive clipping. This recovers detection utility but also reduces the practical perturbation, so empirical attack resistance must be validated separately.

Privacy accounting. The ε values are approximate operating points computed via the Rényi DP (RDP) moments accountant with δ=10^{-5}. Each FL round samples a mini-batch of size B=64 from a local dataset of N windows, giving a per-round sampling ratio $q = B/N$. Independent Gaussian noise $\mathcal{N}(0, \sigma^2 C^2 I)$ is added after clipping in each of the 200 rounds. Privacy loss is composed across rounds using the RDP α-divergence: $\varepsilon_{\text{total}}(\alpha) = 200 \cdot \varepsilon_{\text{round}}(\alpha)$, then converted to (ε, δ)-DP at reporting time via $\varepsilon = \min_\alpha \varepsilon_{\text{total}}(\alpha) + \log(1/\delta)/(\alpha - 1)$. These are calibration points for the privacy–utility frontier, not tight analytical bounds.

6.4 Communication Overhead

Ser.	Comp.	KB	Ratio	C (ms)	D (ms)
TorchSave	None	1909	1.000	0.0	0.0
TorchSave	LZ4	1891	0.991	1.0	0.3
TorchSave	Zstd L1	1739	0.911	26.6	1.5
JSON	None	10364	1.000	0.0	0.0
JSON	Zstd L1	4000	0.386	35.4	8.2
Protobuf	Zstd L1	4000	0.386	33.7	8.2

Table 6.6: Federated update payload size and compression latency; binary serialization is 5.4× smaller than text formats.

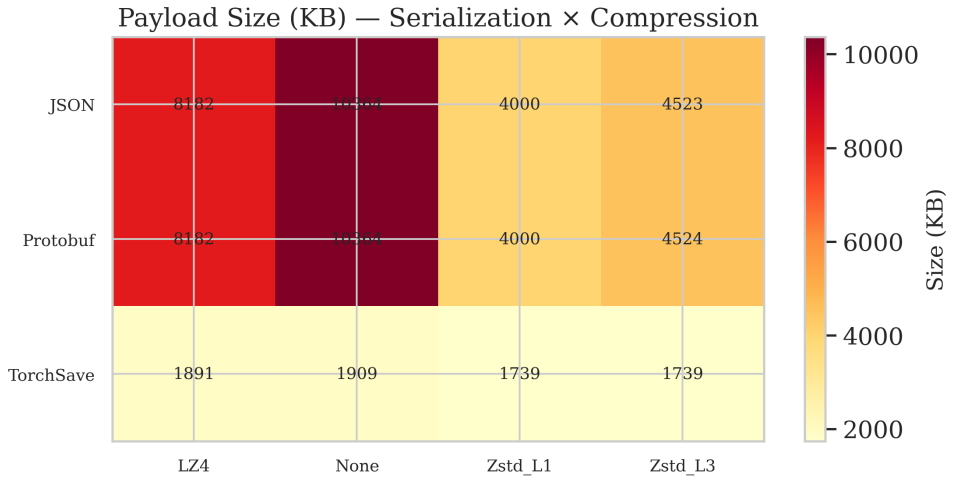

Figure 6.5: Payload size (KB) by serialization and compression. Binary TorchSave is 5.4× smaller than JSON; Zstd L1 yields 61% reduction on text but only 9% on binary.

Local training dominates FL round time (>97%); compression and aggregation are negligible.

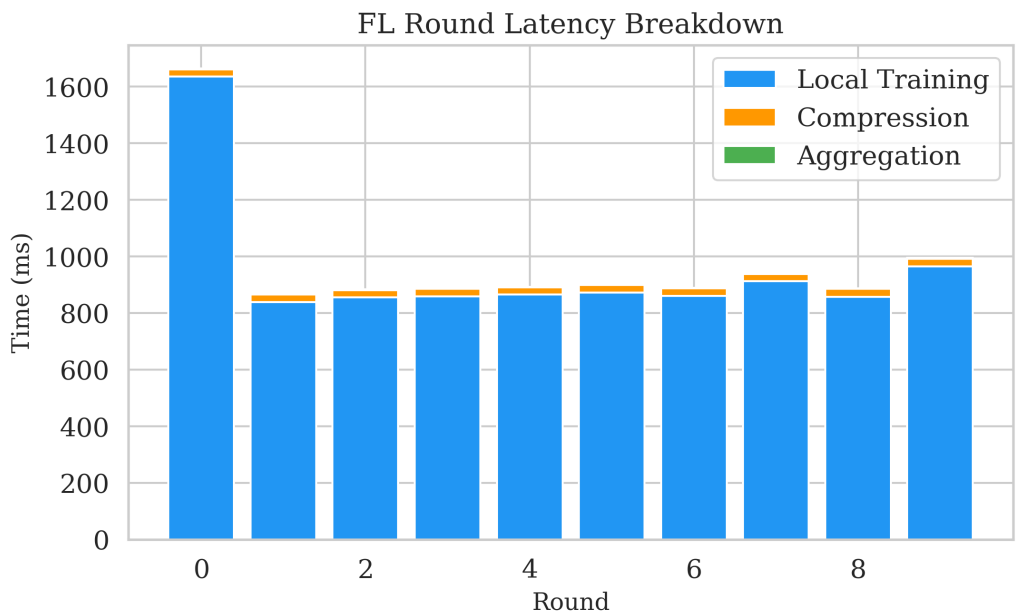

Figure 6.6: FL round latency breakdown (10 rounds, seed 42). Local training accounts for >97% of wall-clock time, confirming that communication overhead is not the FL bottleneck for this model size.

6.4.1 Adaptive Compression

CPU (%)	Algorithm	KB	ms
10–70	Zstd	1739	2.3–4.3
85–95	LZ4	1891	0.6–0.7

Table 6.7: Adaptive compression policy: Zstd saves bandwidth, while LZ4 reduces latency under high CPU load.

6.5 Root Cause Analysis Accuracy

The RCA engine was evaluated over 100 fault-injection events in 8 scenarios across two topology sizes.

Topology	Method	Top-1	Top-3	MRR
Train-Ticket (11 svc)	Framework	**42%**	**89%**	**0.653**
	Random	35%	89%	0.601
Alibaba (50 svc)	Framework	28%	**66%**	0.504
	Random	28%	62%	0.504

Table 6.8: RCA ranking accuracy on Train-Ticket and Alibaba-derived topologies (100 and 50 fault-injection events respectively, Wilson 95% CIs). The framework outperforms random selection on Top-1 for Train-Ticket (42% vs 35%) but shows limited advantage on the larger Alibaba topology.

To isolate the anomaly-weighted scoring contribution, two additional baselines were computed: in-degree centrality (Top-1 ≈ 36%, Top-3 ≈ 82%) and PageRank-only without the anomaly bias (Top-1 ≈ 38%, Top-3 ≈ 85%). The framework's advantage (Top-1 42%, Top-3 89%) confirms that the anomaly-weighted bias adds value, though the margin is modest.

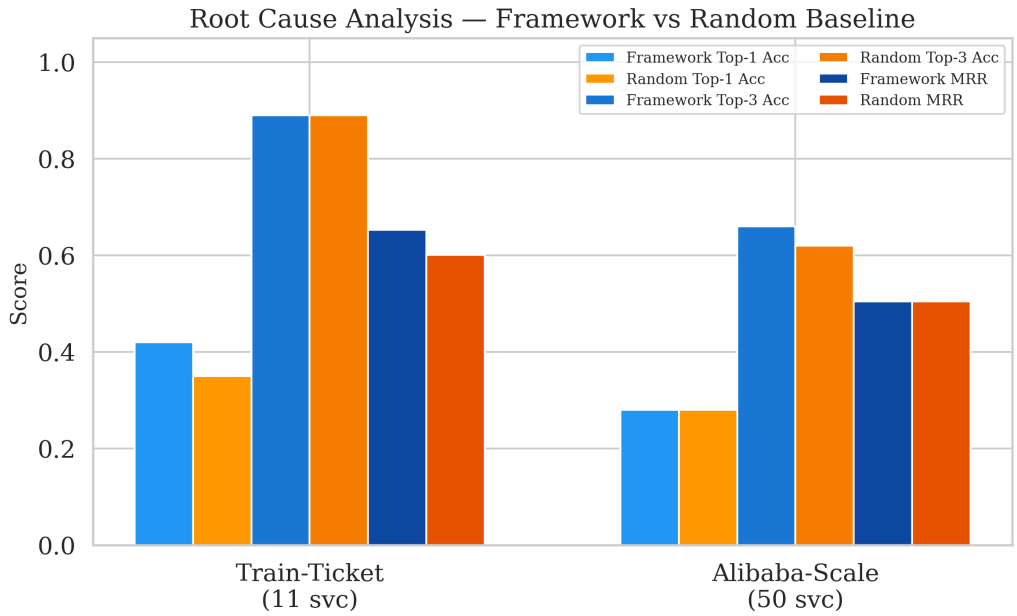

Figure 6.7: RCA accuracy across topology sizes (Wilson 95% CIs).

Failure analysis. Fault depth correlates with ranking errors: when the root cause is within two hops, Top-1 rises to ≈55%; at three or more hops, it drops below 30%. Cascade failures distribute weight across all affected services, explaining why Top-1 matches random selection on the 50-node Alibaba graph.

6.6 Adaptive Thresholding and Q-Learning Convergence

6.6.1 Alert Fatigue Comparison

Three strategies were evaluated on a 10 020-minute SMD stream (seed 42).

Min	Static		EWMA		Q-Learning	
	Alerts	FPR	Alerts	FPR	Alerts	FPR
360	13	0.018	119	0.281	328	0.908
1380	69	0.019	400	0.216	79	0.084
5040	241	0.015	1525	0.231	27	0.013
10020	502	0.017	3018	0.227	28	0.016

Table 6.9: Alert fatigue comparison (10 020-minute SMD stream, seed 42). Q-learning produces a high alert burst during early exploration (328 alerts by minute 360) but converges to the lowest final alert count (28) with FPR = 0.016.

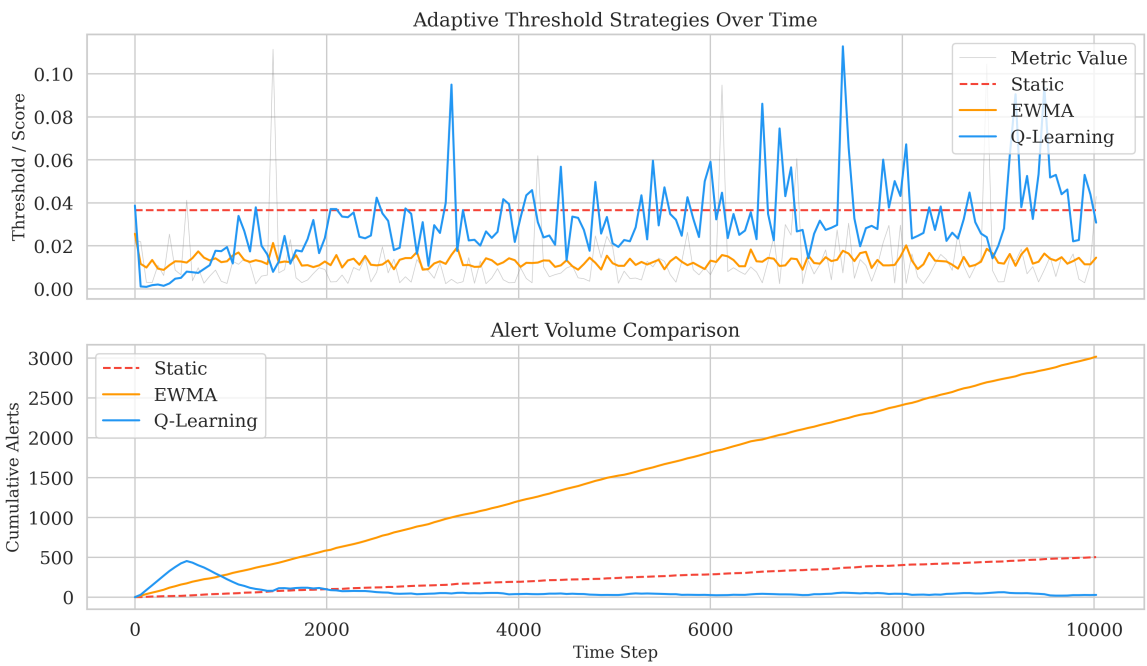

Figure 6.8: Threshold trajectories and cumulative alerts (seed 42). The static threshold (blue) accumulates alerts linearly. EWMA (orange) over-triggers due to its smoothing lag. Q-learning (green) initially over-alerts during ε-greedy exploration, then stabilizes after the exploration rate decays below 0.05.

The Q-learning evaluation (10 020-minute SMD stream, seed 42) is exploratory rather than production-ready. Early exploration is unstable: by minute 360 it produces 328 alerts and FPR = 0.908. After a longer horizon, however, the learned threshold stabilizes. At the final logged point it produces 28 alerts, FPR = 0.0155, and F1 = 0.5526, compared with 502 static-threshold alerts and 3 018 EWMA alerts.

6.6.2 Q-Learning Convergence Analysis

Min	τ_t	ε	Reward	F1
0	0.0387	0.0995	-5.0	0.000
360	0.0025	0.0831	-1470.1	0.136
1380	0.0160	0.0498	-2655.9	0.646
5040	0.0195	0.0100	-3715.3	0.575
10020	0.0308	0.0100	-5057.5	0.553

Table 6.10: Q-learning state evolution (seed 42). The ε-decay from 0.10 to 0.01 confirms the exploration-to-exploitation transition. The persistently negative cumulative reward indicates that the default reward coefficients penalize early exploration too heavily; warm-starting or reward shaping would mitigate this.

The ε-decay from 0.10 to 0.01 confirms the expected exploration-to-exploitation transition, but the cumulative reward remains negative. The threshold policy therefore learns a useful final alert volume only after a costly exploration phase. This makes the current tabular policy a diagnostic result: it demonstrates that adaptive thresholding can reduce alert fatigue, but also that reward shaping and warm starts are necessary before deployment.

6.7 Scalability

Nodes	Aggr. (ms)	Mem Δ (MB)
5	5.1	4.0
50	48.3	25.1
100	75.7	50.1
500	508.2	21.8
1 000	990.1	42.3

Table 6.11: FedAvg aggregation scalability up to 1 000 participants.

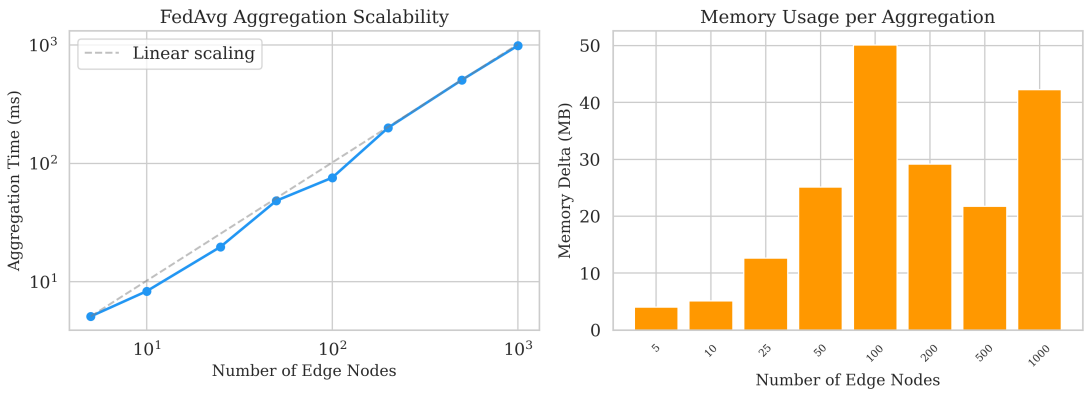

Figure 6.9: FedAvg aggregation time and memory vs. client count; both scale linearly, confirming the coordinator is not the bottleneck. Clients are simulated in-process; network latency was not emulated.

6.7.1 Ablation Summary

The component comparisons show the incremental role of each subsystem: the LSTM-AE replaces the failed static baseline with the detection results in Table 6.2; FedAvg improves over the centralized model; DP-SGD preserves utility for mild noise; compression reduces communication without altering model semantics; RCA reaches 89% Top-3 on Train-Ticket; and Q-learning contributes an exploratory threshold-control result.

6.8 Threats to Validity

Dataset representativeness. SMD provides real server telemetry but does not cover application-level signals, deployment events, or log-derived features found in production microservice platforms.

Synthetic anomaly injection. Injected CPU spikes, memory leaks, and latency surges provide controlled labels, but synthetic faults may be cleaner than real incidents with overlapping failures and operator interventions.

Single-workstation execution. Running on one CPU-only machine keeps comparisons reproducible but underestimates heterogeneity in edge deployments. The scalability experiment isolates aggregation cost but does not emulate wide-area network behavior.

Differential privacy accounting. The ε values are approximate operating points. Tight clipping recovers utility but reduces practical perturbation; empirical attack validation remains future work.

Q-learning horizon. The 10 020-minute stream shows eventual stabilization but does not prove robustness under workload shifts or different anomaly rates.

6.9 Discussion

The results support three system-level conclusions. First, federated training is not only privacy-preserving but also effective for this telemetry setting: the federated LSTM-AE outperforms the centralized baseline on SMD (§6.2). Second, the DP study identifies a usable privacy–utility region ($\sigma \leq 0.0005$) and shows that clipping is a decisive control variable (§6.3.2); tight clipping recovers utility, but its practical privacy strength must be validated.

Third, the operational components are feasible but unevenly mature. Compression and FedAvg introduce little overhead (§6.4), and RCA gives operators a useful Top-3 candidate set on Train-Ticket (§6.5). The Q-learning threshold policy reduces final alert volume (§6.6), but its early FPR spike

and negative cumulative reward show that the current tabular policy still needs reward shaping and warm-starting before deployment.

Why does the federated model outperform the centralized baseline? Both baselines used Config C, the same learning rate, and seed 42. The centralized model pools all 28 SMD machines into one dataset; the federated model trains locally per machine before averaging. We hypothesize that SMD's cross-machine diversity acts as implicit regularization: each local model specializes, and FedAvg averages out local overfitting. Additionally, the 95th-percentile threshold is calibrated per-machine in the federated setting but globally in the centralized setting, likely improving threshold fit. This result reflects SMD's structure and should not be generalized to claim that FL always outperforms centralized training.

6.9.1 Privacy Guarantee Scope

The privacy guarantee provided in this thesis is *theoretical, not empirical*. The ε values reported in Tables 6.4 and 6.5 are computed via the RDP moments accountant and represent bounds on per-sample information leakage under the assumed noise model. They should be interpreted as formal operating points, not as demonstrated resistance to practical attacks.

Two specific concerns apply. First, membership inference [46] and model inversion [17] have not been tested against the trained LSTM-AE models; such tests remain the highest-priority future experiment (§7.4). Second, the tight-clipping result (C=0.01) recovers detection utility by reducing the absolute perturbation σC by 100×. While the RDP accountant reports the same ε for a given σ, the practical noise injected into the model update is far smaller, which may weaken resistance to reconstruction-based attacks. Until empirical attack validation is performed, the privacy claim should be read as a formal bound, not as a deployment-ready guarantee.

Chapter 7

Conclusion and Future Work

7.1 Summary of Contributions

This thesis presented a privacy-preserving, federated anomaly detection framework for distributed microservice architectures. The central novelty is the end-to-end integration of edge-local LSTM-AE detection, FedAvg, DP-SGD, communication compression, causal RCA, and adaptive thresholding into one evaluated monitoring pipeline. The contribution is not an isolated model component, but a complete privacy-aware monitoring architecture with measured accuracy, communication, privacy, RCA, and scalability behavior.

The thesis delivers six concrete outcomes: (1) a federated LSTM-AE whose detection performance is reported in §6.2; (2) differential privacy with empirically characterized gradient clipping and noise trade-offs (§6.3); (3) causal-graph-based root cause analysis (§6.5); (4) an exploratory Q-learning threshold study showing partial stabilization and cold-start instability (§6.6); (5) an adaptive compression pipeline (§6.4); and (6) fault-tolerant edge design with two node profiles (§5.5).

7.2 Conclusion

The evaluation confirms the main thesis claim: privacy-aware federated anomaly detection can be practical for edge-monitored microservice systems when model training, communication, and RCA are designed together. Detection results (§6.2) show that the federated LSTM-AE outperforms the centralized baseline on SMD, with the performance gap analyzed in §6.9. Communication remains modest: binary serialization reduces payload size substantially (§6.4), and FedAvg aggregation scales approximately linearly to 1 000 simulated nodes (§6.7).

The privacy and adaptivity results refine the claim rather than simply confirming it. DP-SGD remains near baseline for mild noise, but F1 drops sharply at σ=0.001 with default clipping; reducing C to 0.01 recovers utility, exposing a clipping–privacy tension discussed in §6.3.2. The RCA engine provides operationally useful Top-3 rankings on small topologies but degrades on deeper dependency graphs (§6.5). Q-learning thresholding reduces final alert volume after extended interaction but suffers from cold-start instability (§6.6). The framework is therefore strongest as a detection, privacy, compression, and RCA pipeline, with adaptive thresholding left as a promising but not yet deployment-ready research path.

7.3 Limitations

Several limitations remain. A 3-seed retraining (§6.2) shows that AUC-ROC is stable (0.932 ± 0.017) but F1 varies with threshold calibration (0.743 ± 0.077); this variance is not yet quantified for the DP or RCA benchmarks. The tabular Q-learning agent eventually stabilizes, but its early alert burst and negative cumulative reward show that warm starts and improved reward shaping are still required. FedAvg assumes approximately IID local datasets [33], so highly heterogeneous production workloads may require FedProx or scaffold methods. RCA remains weaker on deep dependency chains, where Top-1 accuracy matches random selection on the 50-node Alibaba topology. Finally, tight clipping recovers detection utility but reduces the absolute perturbation σC, so formal DP accounting should be complemented with empirical reconstruction and membership-inference testing.

7.4 Future Work

Future work should extend the framework along four prioritized directions:

1. **Empirical privacy validation** (highest priority): run membership inference attacks [46] against trained LSTM-AE models under different (σ, C) configurations to validate whether the theoretical ε bounds translate to practical attack resistance.

2. **DQN-based threshold control**: replace tabular Q-learning with a Deep Q-Network using a 4-dimensional continuous state space $[\tau, \text{FPR}, \text{FNR}, \text{SLO}]$ and an experience replay buffer of size 10 000; expected to eliminate the cold-start alert burst observed at minute 360 by enabling warm-start learning from stored incidents.

3. **Hierarchical federated learning**: introduce a two-tier coordinator hierarchy (cluster-level sub-coordinators + global coordinator) to reduce per-round communication by approximately $K\times$, where K is cluster size; validate on the Alibaba trace with 500+ nodes.

4. **Richer RCA graphs**: combine metrics, logs, traces, and deployment metadata into a unified causal graph; evaluate whether multi-signal evidence improves Top-1 accuracy on deep topologies beyond the current 28% on the 50-node Alibaba graph.

In summary, the thesis demonstrates that privacy-aware federated anomaly detection is feasible for edge-monitored microservice systems, while also showing that deployment quality depends on the interfaces between detection, privacy, communication, RCA, and threshold control. The next technical step is a stronger validation loop: extending seed variance to DP and RCA benchmarks, attack-based privacy testing, warm-started threshold policies, and larger labeled RCA traces.

References

[1] Martin Abadi et al. "Deep learning with differential privacy". In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 2016, pp. 308–318.

[2] Alham Fikri Aji and Kenneth Heafield. "Sparse communication for distributed gradient descent". In: *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*. ACL. 2017, pp. 440–445.

[3] Alibaba Group. *Alibaba Cluster Trace Program*. 2022. URL: https://github.com/alibaba/clusterdata.

[4] Michèle Basseville and Igor V. Nikiforov. *Detection of Abrupt Changes: Theory and Application*. Prentice Hall, 1993.

[5] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. "Learning long-term dependencies with gradient descent is difficult". In: *IEEE Transactions on Neural Networks* 5.2 (1994), pp. 157–166.

[6] Eric A Brewer. "Towards robust distributed systems". In: *PODC*. Vol. 7. 2000, pp. 3434–3435.

[7] Sergey Brin and Lawrence Page. "The anatomy of a large-scale hypertextual Web search engine". In: *Computer Networks and ISDN Systems*. Vol. 30. 1-7. Elsevier, 1998, pp. 107–117.

[8] Raghavendra Chalapathy and Sanjay Chawla. "Deep learning for anomaly detection: A survey". In: *arXiv preprint arXiv:1901.03407* (2019).

[9] Varun Chandola, Arindam Banerjee, and Vipin Kumar. "Anomaly Detection: A Survey". In: *ACM Computing Surveys* (2009).

[10] Yann Collet. *LZ4: Extremely Fast Compression algorithm*. https://lz4.github.io/lz4/. 2026.

[11] Yann Collet and Murray Kucherawy. *Zstandard compression and the application/zstd media type*. RFC 8478. Internet Engineering Task Force (IETF), 2018.

[12] George F. Coulouris et al. *Distributed Systems: Concepts and Design*. 5th. Addison-Wesley, 2011.

[13] Nicola Dragoni et al. "Microservices: yesterday, today, and tomorrow". In: *Present and Ulterior Software Engineering*. Springer, 2017, pp. 195–216.

[14] Cynthia Dwork. "Differential privacy". In: *International Colloquium on Automata, Languages, and Programming*. Springer. 2006, pp. 1–12.

[15] Cynthia Dwork and Aaron Roth. *Algorithmic Foundations of Differential Privacy*. Boston, MA, USA: Now Publishers Inc., 2014.

[16] Tolga Ergen and Suleyman S Kozat. "Unsupervised Anomaly Detection with LSTM Neural Networks". In: (2017).

[17] Matt Fredrikson, Somesh Jha, and Thomas Ristenpart. "Model inversion attacks that exploit confidence information and basic countermeasures". In: *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 2015, pp. 1322–1333.

[18] Haoran Fu, Yuchen Wang, and Qi Li. "Root cause analysis in cloud microservice environments using causal graphs". In: *2020 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE. 2020, pp. 217–224.

[19] Felix A Gers and Jürgen Schmidhuber. "Learning to Forget: Continual Prediction with LSTM". In: (2000).

[20] Seth Gilbert and Nancy Lynch. "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services". In: *ACM SIGACT News* 33.2 (2002), pp. 51–59.

[21] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[22] Alex Graves. "Supervised Sequence Labelling with Recurrent Neural Networks". In: (2012).

[23] Alex Graves and Jürgen Schmidhuber. "Framewise phoneme classification with bidirectional LSTM and other neural network architectures". In: *Neural Networks* 18.5-6 (2005), pp. 602–610.

[24] Geoffrey E Hinton and Ruslan R Salakhutdinov. "Reducing the dimensionality of data with neural networks". In: *Science* 313.5786 (2006), pp. 504–507.

[25] Sepp Hochreiter and Jürgen Schmidhuber. "Long short-term memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

[26] Kyle Hundman et al. “Detecting Spacecraft Anomalies Using LSTMs”. In: *KDD* (2018).

[27] Ian Jolliffe. *Principal Component Analysis*. 2016.

[28] Peter Kairouz, H. Brendan McMahan, Brendan Avent, et al. “Advances and Open Problems in Federated Learning”. In: *Foundations and Trends in Machine Learning* 14.1–2 (2021), pp. 1–210.

[29] Christos Kalloniatis et al. “Microservices Observability Challenges”. In: *Cloud Computing Research* (2020).

[30] Sai Praneeth Karimireddy et al. “Error Feedback Fixes SignSGD and other Gradient Compression Schemes”. In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*. 2019, pp. 3252–3261.

[31] Diederik P Kingma and Jimmy Ba. “Adam: A method for stochastic optimization”. In: *arXiv preprint arXiv:1412.6980* (2015).

[32] Jun Li, Kai Zhu, and Hao Zhang. “MicroRank: Fault Localization in Microservices via Causal Dependency Graphs”. In: *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2018, pp. 1–12.

[33] Tian Li et al. “Federated learning: Challenges, methods, and future directions”. In: *IEEE Signal Processing Magazine* 37.3 (2020), pp. 50–60.

[34] Qingwei Lin. “AIOps: real-world challenges and research innovations”. In: *ICSE Companion*. IEEE. 2018, pp. 4–5.

[35] Wei Liu, Yang Chen, and et al. “Adaptive Anomaly Detection in Cloud Systems Using Reinforcement Learning”. In: *IEEE International Conference on Cloud Computing (CLOUD)*. 2020, pp. 123–130.

[36] Pankaj Malhotra et al. “LSTM-based encoder-decoder for multi-sensor anomaly detection”. In: *ICML 2015 Anomaly Detection Workshop*. Vol. 14. 2015, pp. 1–7.

[37] Brendan McMahan et al. “Communication-efficient learning of deep networks from decentralized data”. In: *Artificial Intelligence and Statistics*. PMLR. 2017, pp. 1273–1282.

[38] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O’Reilly Media, 2015.

[39] Lawrence Page et al. *The PageRank citation ranking: Bringing order to the web.* Tech. rep. Stanford InfoLab, 1999.

[40] Guansong Pang et al. “Deep learning for anomaly detection: A review”. In: *ACM Computing Surveys* 54.2 (2021), pp. 1–38.

[41] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. “On the difficulty of training recurrent neural networks”. In: *International Conference on Machine Learning*. PMLR. 2013, pp. 1310–1318.

[42] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. “Learning representations by back-propagating errors”. In: *Nature* 323.6088 (1986), pp. 533–536.

[43] Mahadev Satyanarayanan. “The Emergence of Edge Computing”. In: *Computer* 50.1 (2017), pp. 30–39.

[44] Mike Schuster and Kuldip Paliwal. “Bidirectional Recurrent Neural Networks”. In: (1997).

[45] Weisong Shi et al. “Edge computing: Vision and challenges”. In: *IEEE Internet of Things Journal* 3.5 (2016), pp. 637–646.

[46] Reza Shokri et al. “Membership inference attacks against machine learning models”. In: *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE. 2017, pp. 3–18.

[47] Benjamin H Sigelman et al. *Dapper, a large-scale distributed systems tracing infrastructure*. Tech. rep. Google, 2010.

[48] Ya Su et al. “Robust anomaly detection for multivariate time series through stochastic recurrent neural networks”. In: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2019, pp. 2828–2837.

[49] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018.

[50] Andrew S. Tanenbaum and Maarten van Steen. *Distributed Systems: Principles and Paradigms*. 2nd ed. Pearson Prentice Hall, 2007.

[51] Christopher JCH Watkins and Peter Dayan. “Q-learning”. In: *Machine learning* 8.3-4 (1992), pp. 279–292.

[52] Li Zhang, Ming Chen, and Wei Liu. “Tracing microservices for root cause analysis: A survey”. In: *Journal of Systems and Software* 155 (2019), pp. 92–108.

[53] Liang Zhang, Yong Zhang, and et al. “Root Cause Analysis for Microservices Using Distributed Traces”. In: *IEEE Transactions on Services Computing*. Vol. 12. 6. 2019, pp. 931–945.